Bandits for Online Exploration in Deep Recommenders

Third Year Project: Final Report

Sulay Chaudhry (5567816)
Department of Computer Science
University of Warwick

Supervised by Prof. Florin Ciucu

Third Year, 2025 – 2026

WARWICK
THE UNIVERSITY OF WARWICK

Abstract

Recommender systems are crucial to serving users personalised digital experiences in the modern world, for which offline-trained deep-learning models such as the two-tower architecture have become dominant. These models retrain in batches on fixed schedules, serving a static policy at inference time misaligned with the real-time sequential nature of online recommendation, and are typically optimised to maximise immediate reward. Together, these have been found to gradually reinforce the model towards its own biases in a degenerative feedback loop.

Contextual bandits offer a principled alternative, modelling recommendation as a sequential game in which they navigate the exploration–exploitation tradeoff to reduce uncertainty about the underlying reward environment. Bandits scale poorly, however, to industry-sized corpuses. Linear bandits remain tractable but lack the representation power of deep-learning models, while neural bandits largely remain computationally infeasible. Hybrid systems combining offline deep representation learning with online linear bandit exploration represent a promising but understudied middle ground.

This work implements and evaluates such a hybrid on KuaiRec, whose dense small matrix permits ground-truth offline policy simulation. Eighteen two-tower variants are trained across architecture, negative-sampling, and ablation axes, each paired with LinUCB, Thompson Sampling, and ε-greedy policies on the frozen embeddings. The hybrid performs competitively relative to well-tuned neural bandit performance reported in prior work, and principled UCB- and posterior-sampling exploration substantially outperforms uniform exploration. Crucially, big-matrix validation accuracy correlates poorly with downstream simulation reward, yet the bandits recover comparable reward across all upstream variants – motivating their use as real-time corrections for biased offline training objectives. They worsen all three beyond-accuracy metrics, however, confirming that bandit exploration is instrumental to exploitation rather than a proxy for bias reduction. These findings motivate future work decoupling diversification from accuracy, and richer reward surfaces that genuinely test the stationarity and linear-realisability assumptions underlying current bandit evaluations.

Keywords: recommender systems, two-tower models, contextual bandits, off-policy evaluation, beyond-accuracy, feedback loops, model bias, deep-learning

Acknowledgements

First, I would like to thank my supervisor, Prof. Florin Ciucu, for his support and insights throughout the year. Florin was incredibly patient and understanding, and it was a delight to regularly discuss this work with him. Second, I would like to thank my family, friends, and partner, whose continual care allowed me to keep pushing through this work through difficult spells of an even more difficult year. Their unwavering belief in my ability is something for which my gratitude truly does extend beyond words.

Use of the Batch Compute System

The authors acknowledge the use of the Batch Compute System in the Department of Computer Science at the University of Warwick, and associated support services, in the completion of this work.

Contents

1 Introduction **4**

1.1 Problem outline 4

1.2 Contribution and objectives 5

2 Background **7**

2.1 Two-tower recommenders 7

2.2 Contextual linear bandits 9

2.3 Bandits in RS literature 10

2.4 Evaluation challenges 12

2.5 Accuracy and beyond-accuracy metrics 13

3 Methodology **14**

3.1 Dataset and data split 14

3.2 Two-tower model and validation 15

3.2.1 Tower inputs 15

3.2.2 Tower architecture 16

3.2.3 Training 16

3.2.4 Validation 17

3.2.5 Model variants 18

3.3 Bandits, feature map, and context construction 18

3.4 Policy simulation 19

3.5 Evaluation metrics 20

4 Implementation **21**

4.1 Two-tower training 21

4.2 Negative sampling 22

4.3 Efficient bandit updates 22

4.4 Memory and device choice 22

4.5 Use of the Batch Compute System 23

4.6 Iteration on the model architecture 23

5 Results and Discussion **24**

5.1 Two-tower validation 24

5.2 Small matrix simulation 27

6 Conclusion **34**

6.1 Summary of results 34

6.2 Reflection on project objectives 35

6.3 Difficulties and lessons learnt 35

6.4 Limitations and future work 36

A Two-tower flags and validation plots **42**

A.1 Model flags 42

A.2 Validation plots 44

Chapter 1

Introduction

Recommender systems (RS) are crucial within a modern digital landscape in which billions of users rely on RS-driven platforms, especially social media, as central to their information diets [1]. Although these algorithms have been massively successful in delivering personalised experiences worldwide, studies have consistently shown that they not only reflect but routinely amplify certain biases present in their training data [2]. Such findings have prompted wider sociopolitical concern [3] around the role of modern RS, motivating RS-specific literature that rigorously examines the reliability of these algorithms.

1.1 Problem outline

The state-of-the-art approach in modern RS involves leveraging deep-learning architectures such as the *two-tower* [4, 5] model. These supervised models train on vast user-item interaction histories to learn complex non-linear user and item representations. Training is computationally expensive, and so these models are usually batch-trained offline with retraining windows that can be hourly or even daily. Between updates, they must serve a static policy that cannot update in real time, and their training objectives – typically involving maximising immediate prediction reward or accuracy – skew them to be exploitation-heavy [6].

The lack of real-time adaptation (the model's *update latency*) often manifests as these recommendations being stale, as the models are unable to quickly capitalise on new information. The exploitation-centric behaviour, on the other hand, often leads to disproportionate recommendation of already-popular items, a Matthew effect [7] ("rich gets richer") widely documented in the literature [8, 9]. Such suboptimal recommendations often still receive some positive engagement from users, creating a feedback loop that reinforces the model's biased policy [6].

A mechanism for breaking such loops is *exploration*: ensuring that the RS goes out of its way to not just exploit well-established positives, but to serve recommendations that actively gather new information about user preferences.

Bandit approaches to RS are an attractive solution to both the stale policy and insufficient exploration issues [10]. Bandit algorithms model recommendation as a sequential online (real-time) learning task in which the *exploration-exploitation tradeoff* must be optimally navigated. These algorithms balance their uncertainty around certain options with the importance of exploiting known high-reward options, with provable guarantees in optimising their long-term cumulative reward [11, Ch. 1].

Despite this, bandit RS in reality has struggled with tractability. Non-linear bandits such as NeuralUCB [12] can in principle match the representation power of offline deep RS, but remain computationally infeasible on industry-scale corpuses. Linear bandits such as LinUCB [13] in comparison can remain tractable on large item spaces, especially so if they leverage compression techniques such as similar-item clustering [14]; their drawback is their purely linear representation power.

This motivates a hybrid approach: an offline batch-trained deep-learning model that produces high-quality representations, supplemented by an online linear bandit that augments the static policy with real-time adaptive exploration. A variant of such a hybrid is deployed at YouTube [10] using a two-tower model as the deep representation learner, but with limited evaluation beyond engagement metrics and little rigorous study of the interplay between these two systems.

Two central questions demand further study in particular. First, how does the upstream embedding quality shape the reward that the linear bandit can capture? Second, how does bandit exploration interact not only with immediate reward, but with beyond-accuracy metrics that measure progress towards bias reduction – metrics often absent in both bandit and supervised RS evaluation?

1.2 Contribution and objectives

This project implements and evaluates a hybrid recommender combining a two-tower model for deep representation learning with linear bandit policies that act on top of the learned embeddings during a simulated retraining window. Evaluation is conducted on the KuaiRec [15] dataset, a highly dense RS dataset built specifically for offline policy simulation, in which nearly all user-item interactions (99.6%) can have their reward simulated. This avoids synthetic approaches required to simulate rewards on most other RS datasets, which tend to be highly sparse.

The aim of such work is to more rigorously understand the performance of these hybrid systems: bridging an exploration of the relationship between deep embedding

quality and bandit reward with beyond-accuracy evaluation, in order to recentre concerns around the very bias that motivates such hybrid work in the first place. To this end, this work pursues the following concrete objectives.

1. Demonstration of a high-quality two-tower model implementation on the KuaiRec dataset, serving as a sound baseline embedding learner for the hybrid system.
2. Implementation of LinUCB, Thompson Sampling, and ε-greedy as linear bandit policies acting on the frozen embeddings, alongside greedy and random baselines.
3. Methodologically rigorous simulation of all bandit policies on the frozen two-tower embeddings, using KuaiRec's dense evaluation matrix to observe genuine ground-truth reward without synthetic estimation.
4. Evaluation of all policies on both accuracy metrics (cumulative reward and *regret*, a measure of missed available reward) and beyond-accuracy metrics (Gini coefficient, long-tail item coverage, and average recommendation popularity).
5. Analysis of how bandit exploration and capturable reward interact with upstream embedding quality, and of the tradeoff between reward and bias-reduction metrics across policies.

Chapter 2

Background

This chapter outlines the technical background required for two-tower models and contextual linear bandits, examines the wider applications of bandits in modern RS, and highlights the difficulties with off-policy bandit evaluation. It also emphasises the current tendency of RS to overemphasise accuracy metrics, and points to the need for beyond-accuracy work.

2.1 Two-tower recommenders

Two-tower recommenders sit firmly within the camp of offline, batch-trained supervised RS models. They can be conceptualised as a natural deep-learning extension of a traditional approach in RS: matrix factorisation (MF) [5].

Given a user-item reward matrix $\mathbf{R} \in \mathbb{R}^{m \times n}$ for m users and n items, MF decomposes $\mathbf{R} \approx \mathbf{U}\mathbf{V}^\top$ where $\mathbf{U} \in \mathbb{R}^{m \times d}$ and $\mathbf{V} \in \mathbb{R}^{n \times d}$ are user and item embedding matrices respectively [4, 16]. The rows $\mathbf{u}_i, \mathbf{v}_j \in \mathbb{R}^d$ are learned d-dimensional embeddings such that the predicted reward is their dot product: $\hat{r}(i,j) = \mathbf{u}_i \cdot \mathbf{v}_j$. In optimising for such an approximation, an MF model must place users and items meaningfully in the compressed d-dimensional embedding space. Traditionally, MF performs this decomposition through linear techniques such as SVD [17].

Two-tower models – popularised by the work of Covington et al. [5] and Yi et al. [18] at YouTube – instead learn the decomposition through two separate neural networks, and leverage rich feature vectors – user demographics, interaction history, item metadata – as inputs rather than purely interaction data. Formally, the predicted relevance between a user i with features $\mathbf{x}_i \in \mathcal{X}$ and an item j with features $\mathbf{y}_j \in \mathcal{Y}$ can be described as

$$\hat{r}(i,j) = f_\theta(\mathbf{x}_i) \cdot g_\phi(\mathbf{y}_j) \tag{2.1}$$

where $f_\theta : \mathcal{X} \to \mathbb{R}^d$ and $g_\phi : \mathcal{Y} \to \mathbb{R}^d$ are the user and item towers (neural networks) with parameters θ and ϕ respectively.

Note that although such a model could use a non-linear scoring function – as many deep RS do – between $f_\theta(\mathbf{x}_i)$ and $g_\phi(\mathbf{y}_j)$, two-tower continues to use the dot-product. This retains geometric interpretability of the embedding space, especially if all embeddings are L2-normalised for which dot-product similarity becomes equivalent to cosine similarity. Additionally, the intentional separation of the towers enables precomputation and caching of embeddings, and the low cost of dot-product scoring reduces retrieval from massive corpuses to a nearest-neighbour search [4].

Training is typically done through *softmax cross-entropy loss* [5, 19]. For each user i's positive item j_i^+ to train on, K negative items $\{j_{i,k}^-\}_{k=1}^K$ are sampled; the model scores all $K+1$ candidates, normalises them to a probability distribution via the softmax function, and cross-entropy loss is applied to penalise confidence in negatives while rewarding confidence in the positive. This pulls positive pairs together in the embedding space while pushing negative pairs apart. This can be formalised as the mean loss $\mathcal{L} = \frac{1}{N}\sum_{i=1}^N \ell_n$ for a batch of N users where

$$\ell_n = -\hat{r}(i, j_i^+) + \log\left(e^{\hat{r}(i,j_i^+)} + \sum_{k=1}^{K} e^{\hat{r}(i,j_{i,k}^-)}\right) \tag{2.2}$$

with implementations such as `CrossEntropyLoss` (PyTorch [20]) widely available in deep-learning libraries. The choice of negative sampling strategy significantly affects training dynamics, as discussed in Chapter 3.

Although newer architectures such as IntTower [21] allow cross-tower interaction, and graph neural networks [22] have become a competitive alternative, a traditional two-tower architecture remains an industry-standard baseline for candidate retrieval and generation [4]. Its central limitation in this work is the mismatch between its fixed-interval offline retraining and benefits of real-time exploration [10], as well as the wider feedback loop issue noted by Chaney et al. [6] Between retraining cycles, the model serves a static policy that recommends well-established positives, continues to observe reward, and reinforces its biased policy at the next update. During this degeneration loop, the model is structurally oblivious to real-time reward signals that challenge its latent representations and allow it to more deeply explore user preferences.

This issue can also be conceptualised as a methodological criticism. Two-tower, and the wider dominant supervised RS paradigm, have been highlighted as often treating recommendation as a purely *observational* problem with respect to the interactions they retrain on, when in reality the role a deployed policy occupies is *interventional* [23]. In the aforementioned feedback loop issue, the popularity-biased RS is completely unaware of its own role in shaping its pool of observable interactions.

2.2 Contextual linear bandits

The multi-armed bandit problem is a reinforcement learning problem between a *learner* and an *environment*, played sequentially. At each round $t = 1, \ldots, n$, the learner plays an action (arm) $a \in \mathcal{A}$ from a finite set of K actions and observes a reward $r_t \in \mathbb{R}$. The goal is typically to maximise cumulative reward $\sum_{t=1}^{n} r_t$ over the horizon n [11, Ch. 1]. Central to this is the aforementioned exploration-exploitation tradeoff: insufficient or unprincipled exploration risks settling on a foreseeably suboptimal exploitation policy, while excessive exploration risks insufficient rounds exploiting high-reward actions.

In applying bandits to RS, it is common to enforce certain assumptions on the environment class $\mathcal{E}$ to make learning possible, meaningful, and computationally feasible – especially over the large corpuses seen in RS. The most common of these is a *stochasticity* assumption [11, Ch. 4]: that for each arm a the underlying reward is being sampled i.i.d. from some distribution P_a. The learner's task is to identify and exploit the arm(s) with highest expected reward while minimising exploration cost. This additionally entails a *stationarity* assumption – that reward distributions do not change over time – a limited but accepted assumption by many systems in practice [2]. In the hybrid system studied in this work, stationarity only needs to hold within a frozen retraining window, a much weaker and more practical assumption than requiring it over the entire RS lifespan.

To avoid a separate learning task not only per arm (infeasible for large item catalogues) but per user, *structure* is imposed through *context*. At each round t, the environment reveals a context $c_t \in \mathcal{C}$ prior to the learner selecting an action; in RS, this is information about users and arms represented as feature vectors. A useful assumption is then that the learner has access to a *feature map* $\psi : \mathcal{C} \times \mathcal{A} \to \mathbb{R}^d$ that maps each context c and action a to a d-dimensional feature vector, such that reward r_t is linear in these features:

$$r_t = \boldsymbol{\theta}_* \cdot \psi(c, a) + \eta_t \tag{2.3}$$

where $\boldsymbol{\theta}_* \in \mathbb{R}^d$ is an unknown weight vector and η_t is stochastic noise assumed to be conditionally 1 sub-Gaussian [1] on all available information for the learner prior to picking their action at round t [11, Ch.19]. Notably, it is not a trivial task to determine such a ψ, but supervised RS models – especially the two-tower, for which it actively trains to model last-layer interaction as linear, as seen in Equation (2.1) – are arguably strong candidates.

This is the set-up of the *contextual linear bandit*, historically the dominant bandit in RS [24]. Given $t - 1$ observed rewards, the weight estimate $\hat{\boldsymbol{\theta}}_t$ (posterior) is typically

[1] This is to say η_t is conditionally assumed to have a tail no heavier than $\mathcal{N}(0, 1)$'s.

obtained through ridge regression:

$$\hat{\boldsymbol{\theta}}_t = \mathbf{A}_t^{-1}\mathbf{b}_t \quad \text{where} \quad \mathbf{A}_t = \lambda\mathbf{I} + \sum_{s=1}^{t-1} \psi_s \psi_s^\top, \quad \mathbf{b}_t = \sum_{s=1}^{t-1} r_s \psi_s \tag{2.4}$$

with $\lambda > 0$ as the ridge regularisation parameter.

The traditional three contextual linear bandits employed in practice are LinUCB, Thompson Sampling and ε-greedy.

LinUCB [13] is a deterministic approach to modelling reward uncertainty, using an upper-confidence-bound approach:

$$a_t = \arg\max_{a \in \mathcal{A}} \left[\hat{\boldsymbol{\theta}}_t^\top \psi(c_t, a) + \alpha \sqrt{\psi(c_t, a)^\top \mathbf{A}_t^{-1} \psi(c_t, a)} \right] \tag{2.5}$$

where $\hat{\boldsymbol{\theta}}_t^\top \psi(c_t, a)$ is the estimated reward, while $\alpha\sqrt{\psi(c_t, a)^\top \mathbf{A}_t^{-1} \psi(c_t, a)}$ is an exploration bonus proportional to reward uncertainty, scaled by a hyperparameter $\alpha > 0$.

Thompson Sampling [25, 26] in contrast is a stochastic algorithm that continually samples a weight matrix centred around the latest posterior in a Bayesian manner. At each round, it samples a $\tilde{\boldsymbol{\theta}}_t \sim \mathcal{N}(\hat{\boldsymbol{\theta}}_t, v^2 \mathbf{A}_t^{-1})$ and then picks the arm maximising $\tilde{\boldsymbol{\theta}}_t^\top \psi(c_t, a)$. The variance scaling hyperparameter $v > 0$ controls exploration intensity.

ε-Greedy [27] explores by simply selecting a uniformly random arm with probability ε and exploiting the estimated best arm otherwise. It is a simple baseline demonstrating the benefits of even minimal, non-adaptive exploration.

A useful quantity for evaluation is *cumulative regret* $R_n = \sum_{t=1}^{n}(r_t^* - r_t)$ where r_t^* is the highest per-arm expected reward available at round t. Regret can be more informative than raw cumulative reward as it accounts for the per-round disparity between available and attained reward, especially in cases where users have already been served their highest-reward items.

2.3 Bandits in RS literature

LinUCB was developed by Li et al. [13] for news recommendation at Yahoo, and is one of the earliest applications of contextual bandits to RS. At the time, features were hand-crafted rather than deep latent representations, and as such the expressive power of these features were strong limits to this model's ability to capture non-linear reward patterns. Thompson Sampling, a near-century-old approach [25], was also adapted for contextual bandit RS around the same period [26]. Such a novel bandit turn was largely overshadowed as deep supervised approaches became dominant [4, 28], both exceeding their non-linear representation power and offering dramatically lower inference cost and scalability for the massive corpuses seen in modern RS.

In more recent literature, bandits largely have not been seen for massive item pools in a way that is competitive with the purpose that two-tower – as a retrieval model – can achieve in acting over an entire, massive item corpus. They predominantly have been deployed as re-rankers [14] operating over much smaller candidate pools downstream of a retrieval model, limiting the potential for corpus-wide exploration and real-time retrieval adaptation. A bandit purely limited to re-ranking 100 or so candidates – where all candidates come from a highly biased model – will likely struggle to mitigate a strong feedback loop.

For instance, a more recent attempt to scale neural bandits for RS is seen by Zhu and Van Roy [29], who propose leveraging an epistemic neural network for efficient Thompson Sampling. Their system impressively demonstrates an end-to-end neural bandit that can retrain its embeddings in real-time, but their scalability test is on the MIND [30] news dataset, for which their bandit purely takes the position of a re-ranker. It is served an 'impression window' of maximally only a few hundred pre-selected news items, and must simply predict click-through rate. This again stops short of demonstrating viability for a bandit to scale to the retrieval stage.

The Neural-Linear [31, 32] family of bandits is a direct motivation for the architecture studied in this work, decoupling periodically retrained deep representation learning from a real-time shallow linear exploration head. Riquelme et al. [31] show that such a system can potentially match or even exceed the regret of end-to-end neural bandits at much lower inference cost. Similarly, however, the deep representation model is one purely built and trained for the downstream linear bandit task, and the datasets are largely synthetic, non-RS, or do not concern massive arm spaces.

Song et al. [14] in contrast demonstrate that linear bandits can in principle act over the entire item catalogue. Their work utilises a hierarchical clustering of the item space to allow bandits to explore and exploit across the whole corpus, without the computational cost of computing estimated reward and uncertainty for every single item each round. This is a rare work arguing for and demonstrating corpus-wide bandits as a direct means of reducing the feedback loop issue. Their evaluation relies on an IPS-based [23] simulator to synthetically estimate rewards from the bandit policies rather than actual ground truth reward; this is a commonly accepted methodological compromise in wider bandit RS, as detailed further in Section 2.4. The approach lacks an upstream retrieval model, however, leaving the result closer to pure bandit RS than to the hybrid architectures that dominate production.

The dominant influence on this project is the Online Matching system deployed at YouTube by Yi et al. [10], the closest published architecture to the one studied here. A two-tower retrieval model is trained offline and its embedding space is discretised into a fixed number of user clusters; a sparse bipartite graph is then built between user clusters and candidate items to reduce the per-user arm space. Additionally, the

authors also introduce a novel Diag-LinUCB variant to allow for even lower latency bandit updates as required in YouTube's setting, a consideration beyond this work. The system is deployed at the candidate generation level rather than as a re-ranker, but experimentally the authors only show moderate A/B-tested gains (+0.15% general engagement, +8.33% fresh content engagement) of such a system within a single fixed UI slot; this leaves the question of the system's generalisability, as it is not independently reproducible. Additionally, the work does not investigate the upstream influence of the two-tower model's quality.

2.4 Evaluation challenges

Offline evaluation of bandit RS policies requires simulating the sequential decision process on logged data. The core challenge is that nearly all RS datasets are highly sparse: in a massive item corpus, each user has interacted with only a small fraction of items. A bandit choosing an arm for which no logged reward exists cannot be evaluated without counterfactual methods such as inverse propensity scoring (IPS) or doubly-robust estimators [23]. These methods introduce assumptions and confounding variables that limit the reliability of the results, especially for policies that deviate heavily from the available ground truth [33].

Another issue is that such synthetic approaches need propensities – or the probability that the user was served an item under the dataset collection policy – which are often unavailable (as in the aforementioned MIND [30] dataset), leading researchers to have to then estimate these propensities; this introduces additional bias. The *Open Bandit Dataset* [33] is a fashion e-commerce dataset and one of the only RS datasets to actually include propensities, but it suffers again from the density issue, and cannot fully overcome a reliance on synthetic reward signals.

KuaiRec [34] is the only known RS dataset to overcome this challenge by providing a *small matrix* of 1411 users and 3327 items with 99.6% interaction density, constructed specifically for offline policy evaluation. This enables querying ground-truth reward for essentially any bandit policy without synthetic reward estimation. The accompanying *big matrix* (7176 users, 10728 items, approximately 16% density) serves as a typical sparse interaction history for pre-training. The tradeoff is scale, however, as an industry-scale dataset like MIND has comparatively 160,000 items and 1 million users.

Even with KuaiRec's density, some residual counterfactual concerns remain. The logged interactions were collected under a different policy, in a different order, over a multi-month period. Sequential dependencies – where a user's response to item B depends on having just seen item A – cannot be captured by querying the static matrix. This is an accepted limitation of KuaiRec-based evaluation; it is eclipsed by the stationarity assumption already made by the bandits, and is considerably less

severe than the synthetic estimation required by sparser datasets. It is also important to note that supervised RS often sidesteps this concern entirely by not evaluating policy deployment at all [35]; working within an imperfect but genuine evaluation framework represents meaningful progress.

2.5 Accuracy and beyond-accuracy metrics

A central argument of this work is that exploration should be evaluated not only by its contribution to reward maximisation, but by its effect on the biases that motivate exploration in the first place. Both supervised and bandit RS evaluation in the wider literature is dominated by accuracy metrics – Recall, NDCG, CTR, cumulative reward, regret – which fail to capture popularity concentration or long-tail suppression [8]; this includes Yi et al.'s [10] work on the Online Matching system. To tackle the feedback loop issue, evaluation must not purely focus on immediate reward and must look at whether the newly proposed policy actively plays into system biases or reduces them, and to what extent.

To this end, this work adopts three metrics – the *Gini Coefficient*, *Long-Tail Coverage*, and *Average Recommendation Popularity* – standard in the beyond-accuracy literature [36, 37], defined formally in Methodology. The literature additionally emphasises qualitative traits such as explainability and interpretability; this is made much easier by two-tower and its transparent embedding space.

Chapter 3

Methodology

This chapter presents the design of the hybrid system. The minimal literature on two-tower–bandit hybrids and the breadth of literature spanning supervised RS and contextual bandits made an iterative research process essential. Rather than committing upfront to a fixed design, work proceeded through repeated cycles of literature review, prototype implementation, and empirical analysis: an idea read in one paper would prompt an experiment, the results of which would expose gaps in understanding or motivate further reading. This was particularly important in determining the depth of two-tower training required to support meaningful bandit behaviour, in disentangling beyond-accuracy effects from accuracy-driven optimisation, and in understanding how upstream embedding quality propagates downstream.

3.1 Dataset and data split

The choice of KuaiRec [15] over alternatives such as MIND [30] or MovieLens [38] is driven by the evaluation challenges outlined in Section 2.4. MIND constrains bandits to re-ranking a pre-selected pool, and MovieLens-style sparse datasets force heavy reliance on synthetic reward estimation. As the only highly dense RS dataset of its kind, KuaiRec accepts a tradeoff in scale for evaluation rigour. The scalability of linear bandits over industry-scale corpuses is established by prior work [10, 14], so this work isolates the contributions of the hybrid system without conflating the effects of upstream clustering or scalability methods.

KuaiRec contains two interaction matrices. The *big matrix* comprises 7176 users and 10728 items at 16.3% interaction density, reflecting a typical sparse RS dataset and used for two-tower training. The *small matrix* comprises 1411 users and 3327 items at 99.6% density and is used for bandit simulation. All small-matrix users and items appear individually in the big matrix – though not under the same logged interactions

– and the two-tower is never exposed to small-matrix interactions during training, preventing data leakage.

KuaiRec does not contain an explicit binary reward signal. The primary interaction signal is `watch_ratio`, the time spent watching a video relative to its duration. Following the baseline suggested by Gao et al. [15], a positive reward is defined as `watch_ratio` ≥ 2.0, indicating that the user watched the video in full at least twice — a strong denoised signal of interest. A robust binary signal is necessary for the two-tower, which typically learns much higher quality embeddings through a thresholded contrastive loss than pure pointwise regression, especially on a noisy signal [19].

The big matrix is filtered to interactions meeting this threshold, deduplicated, and split 80%/20% into training and validation. The split is per-user stratified to ensure validation is not confounded by cold-start effects; users with fewer than five interactions (none of which appear in the small matrix) contribute entirely to training. All experiments use a fixed random seed for full reproducibility.

Rewards are queried from the small matrix out of collection time order, treating the matrix as a static lookup table. This is justified on three grounds. First, it extends the stationarity assumption already inherent to both the bandits and the two-tower's contrastive objective. Second, time-dependent sequential rewards are not meaningfully recoverable from logged data without modelling KuaiRec's logging policy explicitly – a task better suited to simulators such as RecSim [39] or KuaiSim [40] that lie beyond this work's scope. Third, the static framing isolates the time-independent question of interest: whether a bandit can uncover information about user preferences neglected by an exploitation-centric two-tower policy.

3.2 Two-tower model and validation

The two-tower model serves as the offline embedding learner for the hybrid system. After training on the big matrix it is frozen, and its towers produce all user and item embeddings used as input features for the downstream bandits. To study the relationship between embedding quality and bandit performance (Objective 5), a suite of model variants is trained.

3.2.1 Tower inputs

The user tower accepts a user ID alongside 30 side features from KuaiRec's `user_features.csv`: 3 binary activity indicators, 18 anonymised one-hot categorical features, 5 bucketed engagement-range features, and 4 numeric features (follower, fan, and friend counts; and registration age). The user ID is mapped to a 32-dimensional

dense vector via a learnable embedding table (`nn.Embedding`). Each categorical feature is embedded through its own table with width $\min(\lceil \sqrt{|\text{vocab}|} \rceil, 16)$, preventing any single side feature from dominating the representation, with an additional "unknown" category to handle missing values. Numeric features are passed through $\log(1 + x)$ to compress long tails before z-score normalisation. All embedded categorical and normalised numeric features are concatenated with the ID embedding.

The item tower accepts an item ID and a 31-dimensional multi-hot category vector from `item_categories.csv`. The ID is embedded identically (32 dimensions); the multi-hot vector, being already real-valued, is projected to 16 dimensions via a linear layer rather than an embedding table.

3.2.2 Tower architecture

In the default model, both towers share a feedforward architecture: two hidden layers of widths 512 and 256 with ReLU activations and dropout $p = 0.2$, followed by a final linear layer producing a d-dimensional embedding (default $d = 32$). L2 normalisation is applied to outputs, projecting them onto the unit hypersphere so that dot-product similarity becomes cosine similarity. This prevents the model from learning a distorted embedding space that exploits vector magnitudes rather than meaningful angular relationships.

3.2.3 Training

Training uses softmax cross-entropy loss as in Equation (2.2) over $K = 1024$ negatives per positive interaction. For each positive in the batch, dot-product scores against the positive and K negatives are divided by a temperature parameter $\tau = 0.07$ before the softmax; this sharpens the probability distribution, increasing the gradient signal for discriminating between similar items. The model is trained for 100 epochs with the AdamW optimiser at learning rate 10^{-3}, weight decay 10^{-2} and batch size 1024.

Negative sampling strategies

The choice of negative sampling strategy substantially affects the geometry of the learned embedding space [19, 41]. Six strategies are implemented as comparative variants:

- **Uniform** (default): K items are sampled uniformly at random from the catalogue and shared across the batch.

- **In-batch**: each user's positive item acts as a negative for all other users in the batch, with the logQ correction of Yi et al. [18] applied to debias model convergence from the batch distributions.
- **User-uniform**: per-user negatives sampled uniformly from each user's known non-positives (unseen or known negative).
- **Score-weighted**: per-user negatives drawn proportionally to current model scores, implementing online hard negative mining with respect to the model's evolving embedding space.
- **Watch-ratio banded**: per-user negatives sampled from `watch_ratio`-defined hardness bands, treating items the user actively disliked as harder negatives than items they have not seen.
- **Popularity-weighted**: negatives sampled proportionally to item popularity in the training set.
- **Full-softmax**: full cross-entropy loss against the entire item catalogue per positive; the other models generally aim to approximate this a gold standard, as it is not feasible on massive datasets.

3.2.4 Validation

Validation is performed at the end of each epoch by computing corpus-wide Recall@K and NDCG@K for $K \in \{10, 50, 100, 200\}$ across all validation users, with training-set positives masked from the score matrix to prevent metric inflation from items the user has already consumed. Per-user Recall@K measures the fraction of a user's validation positives appearing in the top-K recommendations:

$$\text{Recall@K} = \frac{|\{\text{relevant items in top-}K\}|}{|\{\text{all relevant items}\}|}. \tag{3.1}$$

NDCG@K (Normalised Discounted Cumulative Gain) additionally rewards placing relevant items at higher ranks:

$$\text{NDCG@K} = \frac{\text{DCG@K}}{\text{IDCG@K}}, \quad \text{DCG@K} = \sum_{i=1}^{K} \frac{\text{rel}_i}{\log_2(i+1)}, \tag{3.2}$$

where $\text{rel}_i \in \{0, 1\}$ is the binary relevance of the i-th ranked item and IDCG@K is the ideal DCG achievable. The model checkpoint with highest mean per-user Recall@50 is retained for downstream use; this serves as implicit early stopping.

3.2.5 Model variants

To study how embedding quality shapes bandit performance, variants are trained along four axes: (i) tower *architecture* (hidden widths, depth: `narrower`, `wider`, `deeper`, `shallower`); (ii) *negative sampling* strategy (the seven strategies above); (iii) component *ablations* (`no-sidefeats` disabling side features; `no-relu` replacing non-linearities with linear projections; `no-l2` disabling output normalisation; `id-only-mf` bypassing the towers entirely as an MF-style baseline; `weighted-loss` weighting the cross-entropy by `watch_ratio`); and (iv) *output dimension* ($d \in \{16, 64\}$ in addition to the default 32). Full per-variant CLI configurations are listed in Appendix A.1.

3.3 Bandits, feature map, and context construction

After training, user embeddings $\mathbf{u}_i \in \mathbb{R}^d$ and item embeddings $\mathbf{v}_j \in \mathbb{R}^d$ are computed for all users and items in the small matrix using the frozen towers. Following the feature map formalism of Lattimore and Szepesvári [11, Ch.19] and the design choices of Zhu and Van Roy [29], the bandit context for a user-item pair is constructed as

$$\psi(\mathbf{u}_i, \mathbf{v}_j) = \text{concat}(\mathbf{u}_i, \mathbf{v}_j, \mathbf{u}_i \odot \mathbf{v}_j) \in \mathbb{R}^{3d}, \tag{3.3}$$

where $\odot$ denotes the Hadamard (element-wise) product. The Hadamard term enables the linear model to capture cross-feature interactions between user and item embeddings without the computational cost or overfitting risk of a full outer product.

Each linear bandit policy maintains a single *shared* weight estimate $\hat{\boldsymbol{\theta}}_t \in \mathbb{R}^{3d}$ across all users, rather than per-user disjoint models. This is justified because $\mathbf{u}_i$ already encodes user identity within the context, and a per-user model would learn incredibly slowly given limited per-user rounds. Reward is assumed approximately linear in the feature map, seen prior as Equation (2.3), with the $\hat{\boldsymbol{\theta}}_t = \mathbf{A}_t^{-1}\mathbf{b}_t$ ridge regression updates following Equation (2.4).

Six policies are evaluated on top of this setup, given a user $\mathbf{u}_t$ at time t:

- **Greedy**: serves items in descending order of dot-product similarity $\mathbf{u}_t \cdot \mathbf{v}$ for all available $\mathbf{v}$ – the two-tower's natural static deployment policy and the primary baseline.
- **Random**: samples uniformly over available items.
- **Popularity**: serves items in descending order of their observed popularities (aggregate `watch_ratio`) in the big matrix across small matrix users.
- **ε-greedy** ($\varepsilon \in \{0.01, 0.05, 0.1, 0.2\}$): with probability ε select uniformly from all available items; otherwise select $\arg\max_{\mathbf{v}}(\hat{\boldsymbol{\theta}}_t \cdot \psi(\mathbf{u_t}, \mathbf{v}))$ for all available $\mathbf{v}$.

- **LinUCB** ($\alpha \in \{0.1, 0.5, 1.0, 2.0, 5.0\}$): deterministic optimism in the face of uncertainty, picking:

$$a_t = \arg\max_{\mathbf{v}} \left(\hat{\boldsymbol{\theta}}_t \cdot \psi(\mathbf{u_t}, \mathbf{v}) + \alpha \sqrt{\psi(\mathbf{u_t}, \mathbf{v})^\top \boldsymbol{A}_t^{-1} \psi(\mathbf{u_t}, \mathbf{v})} \right) \tag{3.4}$$

  over all available $\mathbf{v}$, adapted from Equation 2.5.

- **Thompson Sampling** ($v \in \{0.25, 0.5, 1.0, 2.0, 5.0\}$): Bayesian posterior sampling, picking

$$a_t = \arg\max_{\mathbf{v}} \tilde{\boldsymbol{\theta}}_t \cdot \psi(\mathbf{u}_t, v) \quad \text{where} \quad \tilde{\boldsymbol{\theta}}_t \sim \mathcal{N}\left(\hat{\boldsymbol{\theta}}_t, v^2 \boldsymbol{A}_t^{-1}\right) \tag{3.5}$$

  over all available $\mathbf{v}$, as seen prior in Section 2.2.

All linear policies use ridge regularisation $\lambda = 1.0$. Note that the available $\mathbf{v}$ depends on interactions available in the small matrix, and the items recommended to the user already. The popularity baseline was not present in the original objectives, but is a useful additional baseline for policy performance.

3.4 Policy simulation

At each simulation round $t = 1, \ldots, n$, a user $\mathbf{u}_t$ is sampled uniformly at random from the 1411 small-matrix users; all policies observe the same user stream to ensure fair comparison. Each policy selects an item for the user, and the ground-truth `watch_ratio` reward is observed from the small matrix. The bandit policies update $\boldsymbol{A}_t^{-1}$ and $\boldsymbol{b}_t$ accordingly. An item, once recommended to a user, is not re-recommended to that user by that policy in subsequent rounds, reflecting the realistic constraint that a user should not be repeatedly shown identical content.

Regret is computed per-policy. At each round, the regret oracle checks the highest `watch_ratio` attainable for items not yet recommended to the current user by the policy; regret is the difference between that `watch_ratio` and the one observed for the policy's chosen item. This controls for the fact that over time, the 'easy' positives are exhausted, providing a better measure of missed opportunity.

Simulations are run across multiple independent seeds. For each seed, a fresh user stream and fresh per-policy random number generators are derived deterministically from a master seed, ensuring that parallelising seeds across separate compute jobs (via Slurm) produces identical results to a single-process run. Results are reported as **mean** $\pm$ **standard deviation** across seeds, with $n = 10000$ rounds and 100 seeds in the main experiments.

An additional experiment was later added to the main one, as seen in Section 5.2 – this is that of assuming that the reward surface for the bandit is also binarised to $\{0, 1\}$ by the `watch_ratio`. For this experiment, all use of reward – including the popularity policy and regret – was adapted accordingly, as also reflected in the codebase.

3.5 Evaluation metrics

Accuracy is measured by cumulative reward (the running total of positive rewards observed) and cumulative regret (the running total of rounds in which reward was available but not captured). A rolling regret rate (windowed moving average of per-round regret) is additionally tracked to visualise convergence behaviour: whether policies' regret rates decrease as they gather information.

Three beyond-accuracy metrics are recorded at fixed intervals of 500 rounds to track their evolution. Let n_j denote the number of times item j has been recommended across all rounds so far, N the total recommendations made, and $|\mathcal{I}|$ the catalogue size.

The *Gini coefficient* over item recommendation frequencies measures concentration:

$$G = \frac{\sum_{i=1}^{|\mathcal{I}|} \sum_{j=1}^{|\mathcal{I}|} |n_i - n_j|}{2|\mathcal{I}| \sum_{j=1}^{|\mathcal{I}|} n_j}. \tag{3.6}$$

A Gini of 0 indicates perfectly uniform recommendation; a Gini of 1 indicates complete concentration on a single item.

Long-tail coverage at the p-th percentile is the fraction of items whose training-set popularity falls below the p-th percentile that have been recommended at least once, where item popularity $\text{pop}(j)$ uses the same notion of popularity as in Section 3.3. The threshold is set at $p = 80$, capturing items in the bottom 80% of the popularity distribution.

Average Recommendation Popularity (ARP) is the mean popularity of recommended items:

$$\text{ARP} = \frac{1}{N} \sum_{t=1}^{N} \text{pop}(j_t), \tag{3.7}$$

where j_t is the item recommended at round t. Lower ARP indicates less concentration on already-popular items.

The diversity-accuracy tradeoff is visualised through scatter plots of final cumulative reward against each beyond-accuracy metrics, to inspect per-policy differences.

Chapter 4

Implementation

The system is implemented as a Python package `deep_linear_bandits` with a four-command CLI (`dlb`) built on Click [42]: `train-tt`, `simulate`, `collate`, and `plot`. The package separates concerns into four modules – `data` for KuaiRec preprocessing and feature construction, `two_tower` for the PyTorch model and training loop, `simulator` for the NumPy bandit simulation, and `plot` for figure regeneration. Every command writes its flags to a `flags.json` file alongside its outputs, so every experiment in this report is reproducible from its command history alone. Dependencies are managed via `uv`; a fixed master seed of 117 is propagated through NumPy, PyTorch, and CUDA, and a separate `DATA_SPLIT_SEED` holds the per-user stratified split fixed across all eighteen training runs.

4.1 Two-tower training

The model is three `nn.Module` subclasses: `UserTower`, `ItemTower`, and `TwoTower`, with the latter wrapping the pair and exposing a forward pass that returns logits in the $(B, 1 + K)$ shape expected by `CrossEntropyLoss`. Training uses modern PyTorch conveniences: `torch.compile` for model optimisation, and `torch.set_float32_matmul_precision('high')` to enable TF32 on supported GPUs. The `DataLoader` uses pinned memory, four worker processes, and `persistent_workers=True` to minimise batch construction delay and accelerate training. All constructor arguments passed to `TwoTower` are additionally pickled alongside the state dict so that `simulate` can reconstruct any trained model from its checkpoint without re-specifying its architecture.

4.2 Negative sampling

The six negative sampling strategies dispatch through a single conditional block in the training loop, with per-strategy helpers returning $(\text{logits}, \text{target})$ in a common format. This isolates strategy-specific logic from the shared forward-pass, loss, and validation logic, and allowed each variant to be tested as an ablation without forking the training code. For in-batch and popularity-weighted negatives, the logQ correction of Yi et al. [18] is implemented by subtracting sampling log-probabilities from the logits before the softmax; for uniform negatives the correction is constant and therefore omitted, as it does not affect the softmax output. Full softmax uses the same code path as in-batch but with all items $\mathcal{I}$ as the candidate pool rather than the batch – feasible on KuaiRec's catalogue size for comparison, but not on industry-scale corpuses.

4.3 Efficient bandit updates

A naive implementation of LinUCB, Thompson Sampling, or ε-greedy would re-invert the regularised design matrix $\mathbf{A}_t = \lambda \mathbf{I} + \sum_s^{t-1} \psi_s \psi_s^\top$ at each round, costing $\mathcal{O}(D^3)$ per step for context dimension $D = 3d = 96$. Each policy instead maintains $\mathbf{A}_t^{-1}$ directly and updates it via the Sherman–Morrison rank-one formula [43]

$$(\mathbf{A} + \psi\psi^\top)^{-1} = \mathbf{A}^{-1} - \frac{(\mathbf{A}^{-1}\psi)(\mathbf{A}^{-1}\psi)^\top}{1 + \psi^\top \mathbf{A}^{-1}\psi},$$

reducing the per-round cost to $\mathcal{O}(D^2)$. Thompson Sampling additionally requires a sampled $\tilde{\boldsymbol{\theta}} \sim \mathcal{N}(\hat{\boldsymbol{\theta}}, v^2 \mathbf{A}^{-1})$, obtained by Cholesky decomposition [44] $\mathbf{L}\mathbf{L}^\top = \mathbf{A}^{-1}$ and reparameterisation $\tilde{\boldsymbol{\theta}} = \hat{\boldsymbol{\theta}} + v\mathbf{L}\mathbf{z}$ with $\mathbf{z} \sim \mathcal{N}(\mathbf{0}, \mathbf{I})$ [45]. Floating-point error occasionally violates positive-definiteness of $\mathbf{A}^{-1}$; this is handled by a fallback $10^{-6}\mathbf{I}$ added before decomposition, with no observed impact on results.

4.4 Memory and device choice

User–item context vectors $\psi(u, v) = \text{concat}(u, v, u \odot v)$ are constructed on demand for each sampled user inside the `recommend` call – a single $(3327, 96)$ matrix for the current user, dot-producted against $\hat{\boldsymbol{\theta}}$ (or $\tilde{\boldsymbol{\theta}}$ for TS) in one vectorised call. Precomputing the full $(1411, 3327, 96)$ context tensor was considered, but it would occupy roughly 3.4 GB. The bandit simulation is otherwise entirely NumPy; it is CPU-bound and requires no gradients, so running it on the Batch Compute System's scarce GPU nodes would be wasteful. Embeddings are produced once from the frozen model under `torch.inference_mode()`, moved to NumPy, and reused across all seeds and policies for a given upstream model.

4.5 Use of the Batch Compute System

Two-tower models are trained on the DCS Batch Compute System's GPU clusters, using a Bash script to automate `train-tt` Slurm job dispatching. Multi-seed simulations are parallelised over a Slurm job array on the CPU clusters via a `-seed-index` flag that runs a single seed per job. Per-job seeds are derived deterministically from a single master seed prior, ensuring that partitioning across many jobs yields identical results to a single-process run. Each job writes a `seed_{i}.npz` of raw per-round outputs; a dependent `collate` job is dispatched automatically with `-dependency=afterok`, so each 100-seed simulation completes without manual intervention. The separate `plot` command regenerates figures from saved metrics alone, and a `--metric-interval` override permits recomputing beyond-accuracy metrics at different checkpoints without rerunning the simulation.

4.6 Iteration on the model architecture

The two-tower architecture went through several refinements before settling on the default reported in Section 3.2. An initial $[1024, 512] \rightarrow 128$ configuration modelled on the YouTube DNN [5] proved too large for KuaiRec's $10,728$-item catalogue, overfitting despite dropout and weight decay. Progressive narrowing to $[512, 256] \rightarrow 32$ produced a strong baseline; the capacity sweep in Chapter 5 confirms that neither wider, deeper, nor narrower variants meaningfully improve on it. The logit temperature $\tau = 0.07$ was added after early runs exhibited flat softmax distributions with weak gradient signal, following the sharpening convention of Yi et al. [18]. The CLI's extensive flags allowed these experiments to be formalised for reproducibility, enabling the sweep of variants available in Chapter 5 with their flags at Appendix A.1.

Chapter 5

Results and Discussion

The hybrid system is evaluated in two stages, mirroring its two-stage training. Section 5.1 examines the upstream two-tower variants on the big matrix validation split; Section 5.2 examines the downstream bandit policies on the small matrix simulation, both in their own right and in relation to the upstream embedding quality.

5.1 Two-tower validation

All eighteen two-tower variants were trained and validated on the stratified 80%/20% big-matrix split for 100 epochs as outlined in Section 3.1, with checkpointing on best Recall@50. Figure 5.1 shows the convergence behaviour of the `default` model (representative of all variants; the remaining curves are deferred to Appendix A.2); Table 5.1 reports per-user mean Recall@K and NDCG@K across all variants.

A consistent expressiveness ceiling emerges across all variants: best Recall@50 and Recall@200 plateau just over 0.23 and 0.44, and best NDCG@50 and NDCG@200 just over 0.18 and 0.26. The best Recall@50 is 0.23743 (`in-batch-neg`), and the best Recall@200 is 0.44519 (`full-softmax-neg`), while all NDCG@K peak under `pop-weight-neg`. The mirrored convergence of Figure 5.1 across every other model (Appendix A.2), suggests the ceiling reflects an inherent limit of two-tower on KuaiRec-Big rather than inexhaustive model tuning.

Further tower capacity fine-tuning has minimal effect – the `default` hidden widths $[512, 256]$ outperform `narrower`, `wider`, `deeper`, and `shallower` on nearly every metric. `wider`'s Recall@50 of 0.23247 approaches but does not exceed `default`'s 0.23447, and `deeper`'s marginal NDCG@10 advantage (0.14705 vs. 0.14607) is within the noise band suggested by its degradation on every other metric. The output dimension variants `dim-16` and `dim-64` fluctuate noisily around `default`, additionally indicating no expressive bottleneck at $d = 32$.

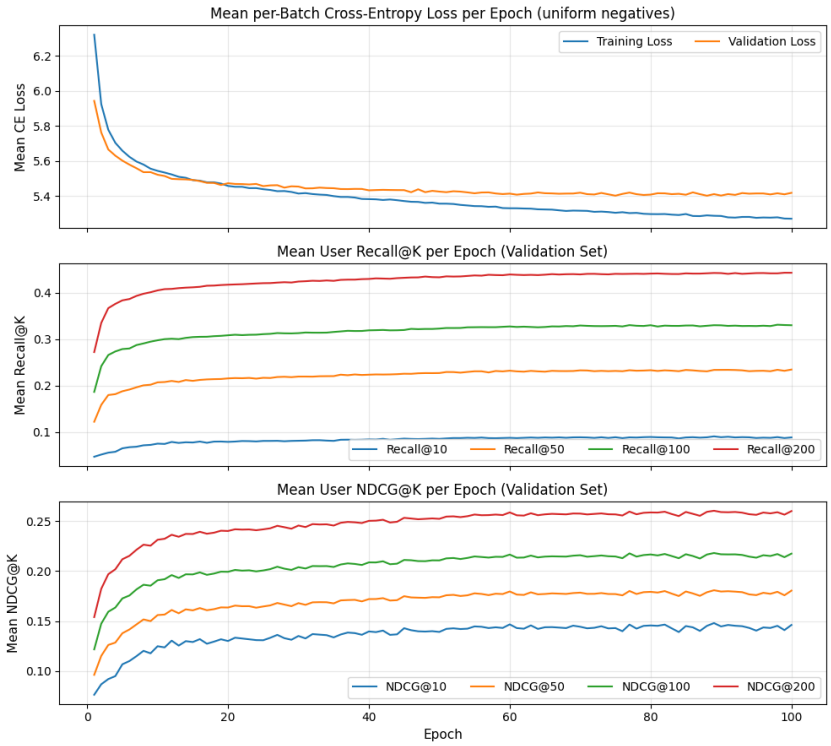

Figure 5.1: Train/val metrics for model `default`

	Mean User Recall				Mean User NDCG			
Model	**@10**	**@50**	**@100**	**@200**	**@10**	**@50**	**@100**	**@200**
`default`	0.08824	0.23447	0.33001	0.44305	0.14607	0.18038	0.21737	0.25990
`narrower`	0.08252	0.22034	0.31402	0.42183	0.13800	0.16982	0.20568	0.24631
`wider`	0.08749	0.23247	0.32645	0.43752	0.13966	0.17496	0.21174	0.25341
`deeper`	0.08760	0.22989	0.32594	0.43818	0.14705	0.17841	0.21549	0.25757
`shallower`	0.08464	0.22598	0.31968	0.42953	0.13561	0.17035	0.20674	0.24814
`in-batch-neg`	0.08940	**0.23743**	**0.33249**	0.44432	0.14727	0.18264	0.21952	0.26150
`user-uniform-neg`	0.08350	0.23219	0.32921	0.44208	0.13611	0.17282	0.21040	0.25282
`score-weight-neg`	0.08513	0.23201	0.32793	0.44064	0.13795	0.17404	0.21127	0.25359
`wr-banded-neg`	0.08618	0.23272	0.32896	0.44317	0.14116	0.17637	0.21374	0.25642
`pop-weight-neg`	**0.09134**	0.23635	0.33005	0.44130	**0.15169**	**0.18427**	**0.22082**	**0.26281**
`full-softmax-neg`	0.08946	0.23710	0.33215	**0.44519**	0.14730	0.18148	0.21843	0.26102
`no-sidefeats`	0.08725	0.23112	0.32435	0.43632	0.14450	0.17738	0.21356	0.25564
`no-relu`	0.08080	0.21471	0.30607	0.41545	0.13363	0.16437	0.19970	0.24063
`id-only-mf`	0.06932	0.20002	0.28661	0.39300	0.10857	0.14409	0.17785	0.21750
`no-l2`	0.08475	0.23087	0.32554	0.43868	0.13516	0.17155	0.20826	0.25062
`weighted-loss`	0.08711	0.22989	0.32242	0.43620	0.14434	0.17761	0.21354	0.25625
`dim-16`	0.09003	0.23382	0.32981	0.44305	0.14642	0.18024	0.21734	0.25967
`dim-64`	0.08974	0.23473	0.33090	0.44197	0.14523	0.17933	0.21640	0.25842

Table 5.1: Recall@K/NDCG@K for all two-tower models on the validation set

The strongest gains come from the negative sampling strategy. The dominance of `in-batch-neg` on Recall@50 and `pop-weight-neg` across all NDCG metrics is notable: Covington et al. [5] frame sampled softmax (SSM) loss as a scalability-aware approximation of full-corpus softmax, yet here `in-batch-neg` (with logQ correction) and popularity-weighted sampling outperform the gold-standard `full-softmax-neg` on most metrics. This is consistent with more recent analysis of SMM loss' effectiveness [19]: popular hard negatives both approximate the full-corpus gradient more efficiently and act as an implicit popularity debiaser, more effective than cross-entropy loss alone at counteracting the popularity skew of the training distribution.

The component ablations confirm the contribution of each part of the model architecture. `no-relu` substantially degrades Recall and NDCG, isolating the role of two-tower's non-linearity; `id-only-mf`, which collapses the architecture to learnable linear MF, degrades further still, quantifying the value of the side features and the deep non-linear towers combined. `no-l2` validates the necessity of the temperature and L2 normalisation; the unboundedness of the logits can also be seen to increase NDCG instability in Appendix A.2. `weighted-loss` – scaling the cross-entropy by `watch_ratio` as in Covington et al. – underperforms `default`, and was not adopted for later experiments.

For reference, a uniform random policy on the validation set yields Recall@50 of 0.00470 (Table 5.2), placing the trained models roughly two orders of magnitude above. While this baseline contextualises the models' retrieval quality, the broader argument is that any well-tuned two-tower variant on this dataset operates near a shared performance ceiling. Critically, this validation is conducted exclusively on the big matrix – the small matrix is not used as a held-out set, since in deployed RS settings access to a dense, off-policy interaction pool is not realistic, and using it would defeat the purpose of treating it as the evaluation environment for the downstream bandit.

Recall Metric	**Mean User Recall@K (Random Policy)**
Recall@10	0.00094
Recall@50	0.00470
Recall@100	0.00941
Recall@200	0.01881

Table 5.2: Random-policy Recall@K baselines on the validation set.

5.2 Small matrix simulation

			Greedy Reward		Best Bandit Reward		
Model	**R@50**[1]	**N@50**[2]	μ	$\pm\sigma$	**Policy**	μ	$\pm\sigma$
`default`	0.234	0.180	14956	79	LinUCB ($\alpha = 2.0$)	25195	494
`narrower`	0.220	0.170	17456	104	LinUCB ($\alpha = 2.0$)	25181	167
`wider`	0.232	0.175	16389	122	LinUCB ($\alpha = 2.0$)	25061	1572
`deeper`	0.230	0.178	14094	96	TS ($\nu = 0.5$)	25087	177
`shallower`	0.226	0.170	18731	102	LinUCB ($\alpha = 2.0$)	25083	647
`in-batch-neg`	0.237	0.183	15359	99	LinUCB ($\alpha = 2.0$)	25244	276
`user-uniform-neg`	0.232	0.173	17643	117	LinUCB ($\alpha = 2.0$)	25116	279
`score-weight-neg`	0.232	0.174	17796	94	LinUCB ($\alpha = 2.0$)	25276	223
`wr-banded-neg`	0.233	0.176	17279	104	LinUCB ($\alpha = 2.0$)	25314	278
`pop-weight-neg`	0.236	0.184	17165	92	LinUCB ($\alpha = 2.0$)	**25394**	487
`full-softmax-neg`	0.237	0.181	16180	87	LinUCB ($\alpha = 2.0$)	25249	290
`no-sidefeats`	0.231	0.177	16201	66	LinUCB ($\alpha = 2.0$)	25345	221
`no-relu`	0.215	0.164	**20257**	95	TS ($\nu = 1.0$)	25154	373
`id-only-mf`	0.200	0.144	16302	81	TS ($\nu = 0.5$)	24808	1070
`no-l2`	0.231	0.172	10728	87	LinUCB ($\alpha = 0.1$)	24910	1285
`weighted-loss`	0.230	0.178	16454	78	LinUCB ($\alpha = 2.0$)	25335	198
`dim-16`	0.234	0.180	16319	72	LinUCB ($\alpha = 2.0$)	25094	511
`dim-64`	0.235	0.179	16161	113	TS ($\nu = 0.5$)	25234	520

Note: static random and popularity baselines yielded rewards 9083 ± 142 and 20711 ± 150 respectively.
[1] Mean per-user Recall@50 on the validation set, from Table 5.1 (3 dp, rounded).
[2] Mean per-user NDCG@50 on the validation set, from Table 5.1 (3 dp, rounded).

Table 5.3: Simulation rewards for all two-tower models and their best bandits

Table 5.3 reports cumulative simulation reward across all eighteen upstream models, comparing the static greedy two-tower policy against each model's best-performing bandit. Three structural observations emerge before the per-model analysis. First, LinUCB is the dominant bandit on most models, with Thompson Sampling occasionally taking the top spot; ε-greedy never wins, reaffirming the value of principled, uncertainty-aware exploration. Second, the dominant hyperparameters ($\alpha = 2.0$ for LinUCB, $\nu \in \{0.5, 1.0\}$ for TS) sit firmly in the middle of the swept ranges, indicating that the optimal exploration-exploitation balance is genuinely tuned rather than saturated at one extreme. Third, the bandit ceiling is remarkably stable across upstream variants: every model except the most degenerate (`no-l2`, `id-only-mf`) converges its best bandit to within a narrow band around $25,000$ cumulative reward, well above the popularity baseline of $20,711$ and the random baseline of $9,083$.

The greedy column reveals a more surprising result: greedy reward correlates poorly with the upstream model's validation Recall@50 and NDCG@50. The ablated `no-relu` model, which has the second-worst Recall@50 of any variant, achieves the highest greedy reward at $20,257$; conversely `default`, near the top of the validation table, sits near the bottom of greedy rewards at $14,956$. Two factors are argued for this. The first is the well-known mismatch between optimising for top-K retrieval metrics – which reward placing relevant items anywhere within the candidate pool – and predicting raw interaction scores at the per-item level, an asymmetry intrinsic to retrieval-stage models [4]. The second, and likely more important, is that the big matrix on which validation is conducted is itself the product of a biased logging policy: fitting it more closely is not equivalent to generalising to the small matrix's near-complete reward surface. A model that fits the big matrix's popularity skew well may simply be inheriting that bias rather than learning a transferable preference signal, and this distinction is invisible to a purely big matrix train/validation split.

The robustness of the bandit ceiling against upstream model degradation is a significant finding. Even `no-l2`, whose greedy reward of $10,728$ is below the popularity baseline, recovers to a bandit reward of $24,910$; `no-relu` – again the model with best greedy performance – only recovers to 25,154. The bandit is largely insensitive to the ranking quality of the upstream embedding model it operates over, provided that the embeddings have still learned enough for small matrix reward to be approximately linear in the combined context. Given again the difficulty of retrieving off-policy logs to evaluate the debiased quality of an upstream model in production settings, this suggests a significant benefit: the bandit can correct for a substantial (potentially unknown) mismatch upstream with real-time deployment. Greedy performance – the model's naive dot product ranking quality – is a mere subset of the learnable $\hat{\boldsymbol{\theta}}$ available to the bandit through its Hadamard context term (Equation (3.3)).

Figure 5.2 shows cumulative reward curves for `pop-weight-neg`; convergence to the

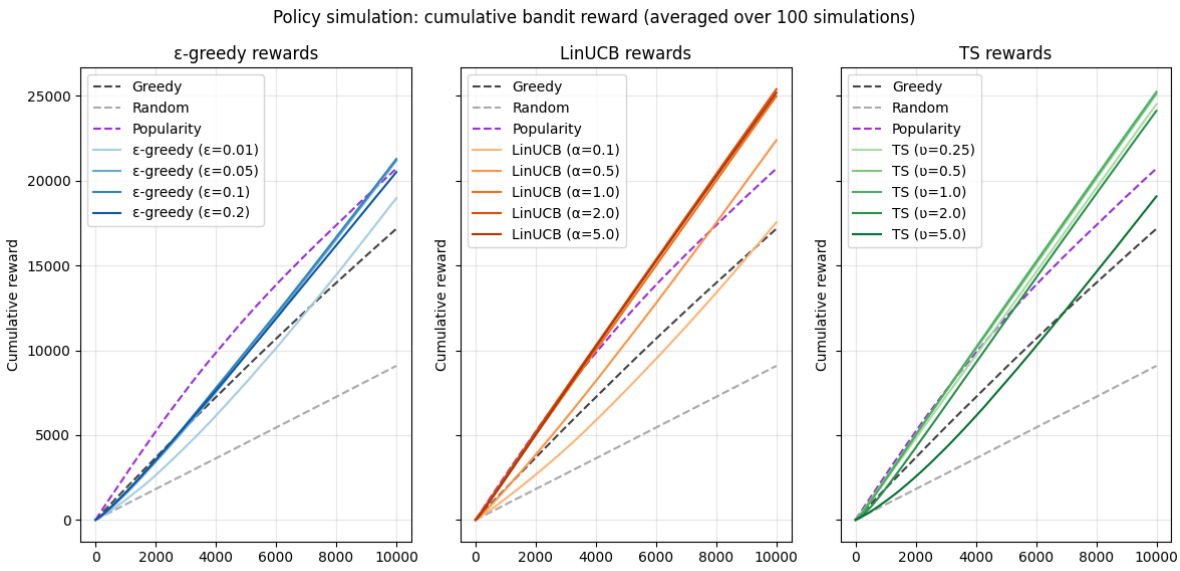

Figure 5.2: Cumulative reward for simulated policies on `pop-weight-neg`

$\approx 25,000$ ceiling occurs well before the $10,000$-round horizon for all linear bandits. These results compare favourably against Zhu and Van Roy [29], who report a well-tuned bandit average of 2.4–2.6 over 6,000 rounds with 10 seeds. The hybrid system here matches or exceeds their reported performance for all but their most aggressive end-to-end neural bandit (ENR), using a linear bandit on offline-trained two-tower embeddings rather than continually retrained representations. Two factors likely contribute to this: the offline two-tower already provides a much more informative starting point than the bandit-only setup of Zhu and Van Roy, who note that integrating offline modelling is left as future work; and the contrastive softmax cross-entropy training of the two-tower likely yields higher quality embeddings than the regression objective they employ.

The robustness across upstream models also raises a subtler question about the geometry of the reward surface. The fact that an MF-style baseline (`id-only-mf`), an architecture ablation (`no-relu`), and a fully-tuned model (`default`) all converge to comparable bandit ceilings suggests two possibilities. First, the binarised reward surface may be sufficiently linearly capturable that the additional expressive power of a richer feature map yields diminishing returns: the bandit's ridge regression on $\psi(\mathbf{u}, \mathbf{v}) = \text{concat}(\mathbf{u}, \mathbf{v}, \mathbf{u} \odot \mathbf{v})$ may already be saturated. Second, the dot-product training objective of two-tower may not exert pressure on the embeddings to encode information exploitable beyond the dot product itself, limiting the extent to which the Hadamard term in ψ can extract additional structure. The higher variance of `wider` (± 1572) and `no-l2` (± 1285) bandit rewards, relative to the typical ± 100–500 band, suggests sparser or less-converged embeddings may retain exploitable structure that the bandit can sometimes succeed in recovering – and conversely sometimes fails to escape due to the misspecification of assuming a perfect feature map and a strictly

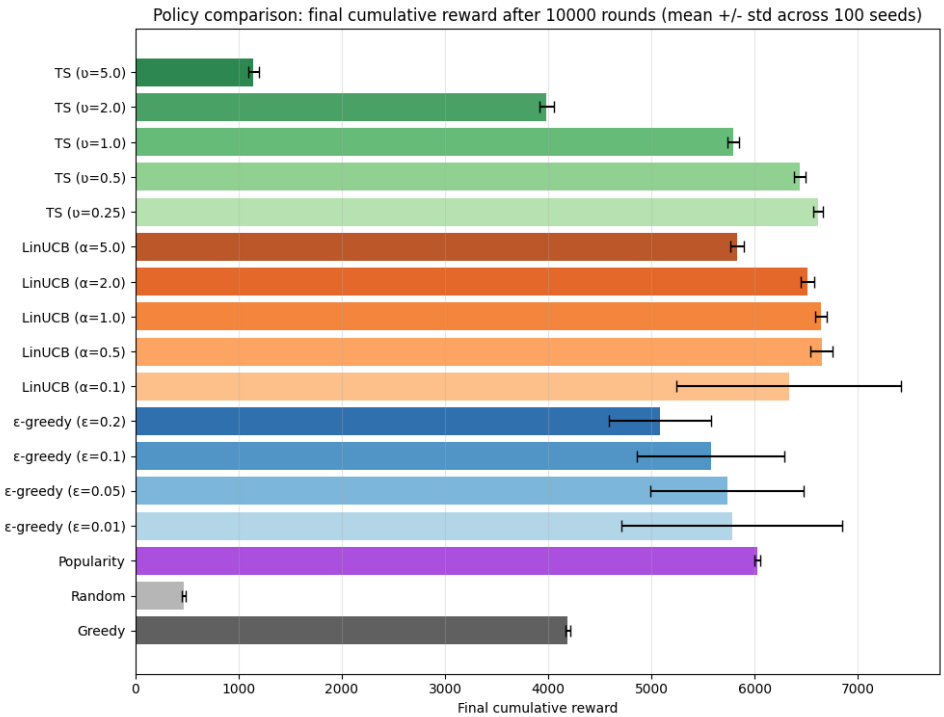

Figure 5.3: Final cumulative reward under a **binary** signal on `pop-weight-neg`

linear environment.

A complementary perspective comes from a binary reward ablation in Figure 5.3, where the reward signal is reduced to $\{0, 1\}$ (matching the embedding learning threshold). Under this restricted information, the bandits only marginally outperform the popularity baseline, suggesting that much of the bandit's gain in the main experiment comes from exploiting a reward landscape in `watch_ratio` that the binary surface flattens. Even so, Figure 5.4 shows that the rolling regret of the popularity baseline climbs steadily over time as it exhausts the front-loaded benefit of always recommending popular items, while the bandit policies' regret rates remain stable: even under the restricted signal, the bandits have learnt structure that the popularity heuristic cannot.

The picture for beyond-accuracy metrics is less favourable. Figure 5.5 plots the diversity–accuracy tradeoff across all policies on `pop-weight-neg`: the best linear bandits cluster at high reward but also high Gini, low long-tail coverage, and high ARP – in fact slightly worse on each beyond-accuracy axis than the static greedy policy. This is unsurprising and expected for LinUCB and Thompson Sampling, both of which are designed to reduce exploration as the posterior tightens around an exploitation-favoured policy: exploration is only pursued to the extent that it improves future exploitation. The bandit's reward orientation is doing exactly what it is designed to do; the framing problem is treating exploration alone as a proxy for bias reduction.

It is notable however that the bandits do not just simply cluster to the popularity

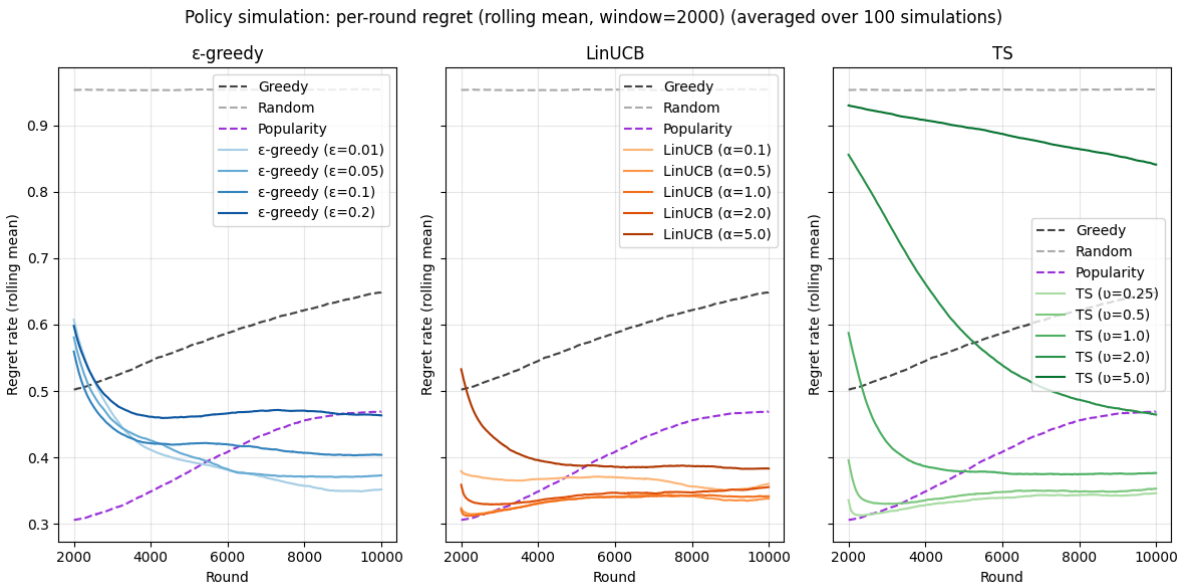

Figure 5.4: Rolling per-round regret under a **binary** signal on `pop-weight-neg`

ceiling ARP that nears 10,000 but instead find exploitation-optimal items to generally converge around a lower ARP around 7000–8000. It is not unreasonable to assume that there will in fact be a reasonable correlation between popularity and model reward; the question then shifts again towards whether it is methodologically sound, or desirable, to purely treat popularity debiasing through the lens of accuracy optimisation.

This points to two directions for work that genuinely targets bias reduction. First, beyond-accuracy objectives could be incorporated directly into the bandit's reward function – though this requires carefully ensuring that the resulting reward space remains linearly separable for the bandit, which is not trivial. Second, the relevant horizon for measuring the debiasing benefit of bandit exploration may extend beyond a single retraining window. The bandit's exploration-aware deployment policy should compound diversification benefits when those interactions are used in two-tower retraining, in line with longer-horizon RL framings of exploration value [46].

Regardless, it is likely that the value of bandit exploration in deep RS is most naturally seen and studied in a multi-cycle retraining pipeline rather than within a frozen window; the main barrier is designing a high-quality experiment to capture this offline with off-policy data, or better yet rigorous research in an online environment.

Finally – while KuaiRec remains the only fully dense evaluation dataset of its kind, and is invaluable for off-policy evaluation – the saturation of bandit performance across fundamentally different upstream models hints that its reward surface may be more linearly capturable, and exploration-averse, than a deployed RS environment would be in reality. Synthetic simulators such as RecSim or KuaiSim are one alternative, but Pires et al. [47] suggest that exploitation-heavy linear models seem to also fit their reward surfaces disproportionately well. This heavily motivates the wider

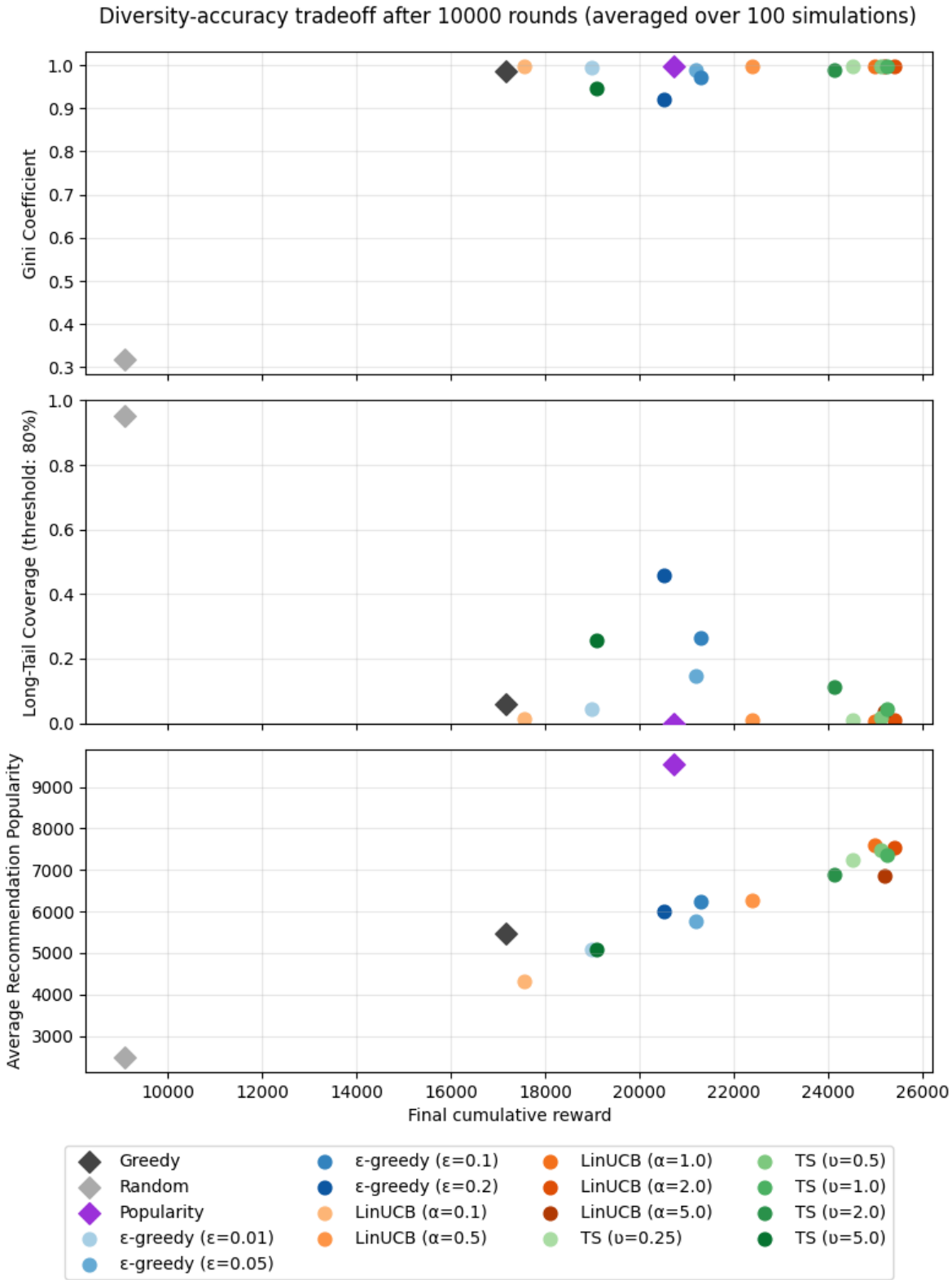

Figure 5.5: Diversity–accuracy tradeoff across policies on `pop-weight-neg`

methodological importance of off-policy evaluation datasets that more rigorously test models' navigation of the exploration-exploitation tradeoff, since current methods may drastically underrepresent the necessity of exploration and the difficult non-linear reward landscapes seen in online deployment. Again, such a task is not trivial; it may be the case that transcending deeply nested assumptions such as stationarity will be essential, meanwhile offline-policy evaluation is still generally grappling with dataset sparsity issues.

Chapter 6

Conclusion

6.1 Summary of results

This work implemented and evaluated hybrid recommender systems combining frozen two-tower embedding models with contextual linear bandit policies acting on the learned embedding spaces. Eighteen two-tower variants were trained across architecture, negative sampling, and ablation axes, each paired with the full bandit suite on KuaiRec's small matrix. LinUCB and Thompson Sampling were the dominant policies, lifting simulation reward to a common ceiling of around $25,000$ across $10,000$ rounds – a per-round average of approximately 2.5, competitive within the 2.4–2.6 range highlighted by Zhu and Van Roy [29]. Much of the online benefit they demonstrate therefore appears capturable through principled offline representation learning, favouring hybrid offline-online systems such as this one as highly viable given the computational demands of real-time neural bandits.

Greedy reward on the small matrix correlated poorly with big-matrix validation Recall@K and NDCG@K, reaffirming that fitting to traditional metrics on sparse biased interaction histories does not entail downstream generalisability to a less biased environment. The resilience of the bandit ceiling even for models whose greedy reward underperformed the popularity baseline suggests an attractive robustness for hybrid linear bandits in deployment, where upstream model bias may not be fully known.

The relative linear realisability of the reward surface, however, supports recent arguments [47] that current off-policy evaluation datasets may underrepresent the difficulty of exploration in deployed RS. The bandits additionally worsened all three beyond-accuracy metrics (Gini Coefficient, Long-Tail Coverage, Average Recommendation Popularity) over the greedy baseline, reinforcing the fact that exploration in the bandit sense is instrumental to exploitation rather than a direct proxy for bias reduction.

6.2 Reflection on project objectives

All five objectives in Section 1.2 were fulfilled: a two-tower implementation reaching the empirical ceiling across a wide sweep of choices, three contextual linear bandit classes with competitive convergent performance, simulation on ground-truth reward aggregated across 100 seeds per model, evaluation spanning both accuracy and beyond-accuracy axes, and systematic upstream-downstream analysis across all eighteen variants. What this project did not – and within scope could not – achieve is an account of the long-term value of bandit exploration across retraining cycles, nor of the bandit's role in reshaping the data distribution seen by the next upstream training pass. The frozen context framing mirrors a standard evaluation protocol in bandit RS, but neglects the feedback loop that partially motivates such hybrid work in the first place.

6.3 Difficulties and lessons learnt

First, the literature on such a hybrid system was incredibly limited; bandits as re-rankers are much more common, while YouTube's *Online Matching* [10] was the only system found to combine two-tower and bandits in retrieval. It runs on a proprietary platform with no code and gained verified only through private A/B testing.

Further, two-tower is often reported with inconsistency – regression versus cross-entropy; with or without logQ correction; and contested views on sampled softmax, which Covington et al. [5] framed as a scalability-driven approximation but recent work [19] and this project find can exceed full softmax performance. Bandit RS is in a similar position, with off-policy evaluation methods varying by paper under differing counterfactual assumptions, and with the viability of many available datasets in question [47].

This made navigating both sides of the literature difficult, especially given the breadth of concepts needed to iterate on the hybrid system. To overcome this, it was found that implementing in parallel with reading was essential – softmax loss discrepancies, and the logQ correction especially, only became clear through empirical inspection – and reproducing prior work at small scale was among the better uses of time, both as a sanity check on the implementation and as a way of exposing claims that did not transfer as cleanly as their papers suggested.

Embedding learning on a dataset whose items cannot be inspected – KuaiRec's videos are not viewable – was substantially harder than it first appeared, and a future project of this kind would benefit enormously from a diagnostic suite built up front. This

would also facilitate building up an intuition for the geometry, incentives, and reward surface attainable.

6.4 Limitations and future work

Methodologically, both the two-tower and the bandit assume stationarity, as does the KuaiRec simulation: true sequential interactions cannot be recovered, for which the role of a bandit in guiding a user towards new preferences in real-time cannot be surveyed. Further, the bandits are stateless, linear, and assume stochasticity: extending this work to survey harsher reward landscapes is crucial to drastically highlight the ceiling of these hybrid systems in deployed RS, in line with Pires et al. [47] Such work would also benefit from surveying adversarial bandits and the wider recent work in deep RL.

Coupling beyond-accuracy objectives directly into the bandit reward, given approximate linearity in the feature map, is an interesting direction. Another is exploring how this performance is affected by the bandit clustering used in scaling to industry-size corpuses [14, 10]; additionally it would be incredibly informative to survey a wide range of hybrid systems, especially across different upstream retrieval models such as GNNs [22] and VAEs [48].

Most importantly, bandit exploration is best measured across multiple retraining cycles, where its compounded effect on upstream training can be captured; this should be a significant object of further work. Finally, a thread throughout this work is that deployed recommenders are as much a data collection policy as much as they are a serving policy – it is crucial to explore additional measures that can be taken by the upstream model to better account for this mismatch.

Author's Assessment of the Project

This project set out to investigate hybrid recommenders bridging the dominant batch-trained supervised paradigm with contextual bandits for online learning, motivated by their proposed use for alleviating bias and degenerative feedback loops at the retrieval stage. This work has successfully affirmed their value on the KuaiRec dataset relative to existing neural bandit work, and highlighted a potential strength in these systems in a resilience to upstream training misspecification and bias, as well as their reduced computational cost. This work also highlighted the importance of decoupling beyond-accuracy and debias work further from exploration used purely as a proxy for accuracy. Such investigation was grounded by a continual look at wider methodological concerns across both supervised RS and off-policy evaluation for bandits, often neglected by the wider literature. An excessive focus on reporting accuracy statistics without regard for downstream mismatch, bias, or methodological transparency, is all too common in not only RS but the wider ML literature, and to this end this work sought to restate the importance of departing from pure data-fitting for the sake of published research.

Others may make use of this work through its extensive CLI, which enables thorough investigation of two-tower–bandit hybrid systems on KuaiRec, and is easily extensible. More importantly, they should use these findings as a motivation for further study on hybrid systems, reward landscapes, and rigour in off-policy evaluation. This project is simply one of many small steps required to both broaden the toolkit of methods available in RS design and sharpen the scrutiny under which systems should be evaluated, especially offline. Further, others may use to extend the discussion here to more rigorously scrutinise the observational assumptions made upstream by these recommenders, and fundamentally challenge narratives around stationarity.

To that end, the stationarity assumptions required throughout this work – due to current limitations in off-policy evaluation, bandits, two-tower, and KuaiRec itself – restrict this work's ability to more thoroughly investigate those considerations at this stage. Further is the much wider range of systems that need comparison, and the need for much harsher reward landscapes. Finally, the long-term effects of the bandit over many retraining cycles very much needs further study, under a rigorously justified evaluative approach.

Bibliography

[1] Sushim Kanchan and Abhay Gaidhane. 'Social media role and its impact on public health: a narrative review'. In: *Cureus* 15.1 (2023).

[2] Jiawei Chen et al. 'Bias and debias in recommender system: A survey and future directions'. In: *ACM Transactions on Information Systems* 41.3 (2023), pp. 1–39.

[3] Qazi Mohammad Areeb et al. 'Filter bubbles in recommender systems: Fact or fallacy - A systematic review'. In: *Wiley Interdisciplinary Reviews: Data Mining and Knowledge Discovery* 13.6 (2023), e1512.

[4] Junjie Huang et al. 'A comprehensive survey on retrieval methods in recommender systems'. In: *ACM Transactions on Information Systems* 44.1 (2025), pp. 1–43.

[5] Paul Covington, Jay Adams and Emre Sargin. 'Deep neural networks for youtube recommendations'. In: *Proceedings of the 10th ACM conference on recommender systems*. 2016, pp. 191–198.

[6] Allison JB Chaney, Brandon M Stewart and Barbara E Engelhardt. 'How algorithmic confounding in recommendation systems increases homogeneity and decreases utility'. In: *Proceedings of the 12th ACM conference on recommender systems*. 2018, pp. 224–232.

[7] Matjaž Perc. 'The Matthew effect in empirical data'. In: *Journal of The Royal Society Interface* 11.98 (2014), p. 20140378.

[8] Anastasiia Klimashevskaia et al. 'A survey on popularity bias in recommender systems'. In: *User Modeling and User-Adapted Interaction* 34.5 (2024), pp. 1777–1834.

[9] Cataldo Musto, Pasquale Lops, Giovanni Semeraro et al. 'Fairness and Popularity Bias in Recommender Systems: an Empirical Evaluation.' In: *DP@ AI* IA*. 2021, pp. 77–91.

[10] Xinyang Yi et al. 'Online matching: A real-time bandit system for large-scale recommendations'. In: *Proceedings of the 17th ACM Conference on Recommender Systems*. 2023, pp. 403–414.

[11] Tor Lattimore and Csaba Szepesvári. *Bandit algorithms*. Cambridge University Press, 2020.

[12] Dongruo Zhou, Lihong Li and Quanquan Gu. 'Neural contextual bandits with ucb-based exploration'. In: *International conference on machine learning*. PMLR. 2020, pp. 11492–11502.

[13] Lihong Li et al. 'A contextual-bandit approach to personalized news article recommendation'. In: *Proceedings of the 19th international conference on World wide web*. 2010, pp. 661–670.

[14] Yu Song et al. 'Show me the whole world: Towards entire item space exploration for interactive personalized recommendations'. In: *Proceedings of the Fifteenth ACM International Conference on Web Search and Data Mining*. 2022, pp. 947–956.

[15] Chongming Gao et al. 'KuaiRec: A fully-observed dataset and insights for evaluating recommender systems'. In: *Proceedings of the 31st ACM International Conference on Information & Knowledge Management*. 2022, pp. 540–550.

[16] Google. *Collaborative filtering | Machine Learning | Google for Developers*. 2025. URL: https://developers.google.com/machine-learning/recommendation/collaborative/basics (visited on 17/01/2026).

[17] Jacob Munson, Breschine Cummins and Dominique Zosso. 'An introduction to collaborative filtering through the lens of the Netflix Prize'. In: *Knowledge and Information Systems* 67.4 (2025), pp. 3049–3098.

[18] Xinyang Yi et al. 'Sampling-bias-corrected neural modeling for large corpus item recommendations'. In: *Proceedings of the 13th ACM conference on recommender systems*. 2019, pp. 269–277.

[19] Jiancan Wu et al. 'On the effectiveness of sampled softmax loss for item recommendation'. In: *ACM Transactions on Information Systems* 42.4 (2024), pp. 1–26.

[20] PyTorch. *CrossEntropyLoss — PyTorch 2.10 Documentation*. 2026. URL: https://docs.pytorch.org/docs/stable/generated/torch.nn.CrossEntropyLoss.html (visited on 05/02/2026).

[21] Xiangyang Li et al. 'Inttower: the next generation of two-tower model for pre-ranking system'. In: *Proceedings of the 31st ACM International Conference on Information & Knowledge Management*. 2022, pp. 3292–3301.

[22] Yiwen Yuan et al. 'ContextGNN: Beyond two-tower recommendation systems'. In: *arXiv preprint arXiv:2411.19513* (2024).

[23] Ziyou Yan. 'Counterfactual Evaluation for Recommendation Systems'. In: *eugeneyan.com* (Apr. 2022). URL: https://eugeneyan.com/writing/counterfactual-evaluation/.

[24] Nícollas Silva et al. 'Multi-armed bandits in recommendation systems: A survey of the state-of-the-art and future directions'. In: *Expert Systems with Applications* 197 (2022), p. 116669.

[25] William R Thompson. 'On the likelihood that one unknown probability exceeds another in view of the evidence of two samples'. In: *Biometrika* 25.3/4 (1933), pp. 285–294.

[26] Shipra Agrawal and Navin Goyal. 'Thompson sampling for contextual bandits with linear payoffs'. In: *International conference on machine learning*. PMLR. 2013, pp. 127–135.

[27] Peter Auer, Nicolo Cesa-Bianchi and Paul Fischer. 'Finite-time analysis of the multiarmed bandit problem'. In: *Machine learning* 47.2 (2002), pp. 235–256.

[28] Shuai Zhang et al. 'Deep learning based recommender system: A survey and new perspectives'. In: *ACM computing surveys (CSUR)* 52.1 (2019), pp. 1–38.

[29] Zheqing Zhu and Benjamin Van Roy. 'Scalable neural contextual bandit for recommender systems'. In: *Proceedings of the 32nd ACM International Conference on Information and Knowledge Management*. 2023, pp. 3636–3646.

[30] Fangzhao Wu et al. 'Mind: A large-scale dataset for news recommendation'. In: *Proceedings of the 58th annual meeting of the association for computational linguistics*. 2020, pp. 3597–3606.

[31] Carlos Riquelme, George Tucker and Jasper Snoek. 'Deep bayesian bandits showdown: An empirical comparison of bayesian deep networks for thompson sampling'. In: *arXiv preprint arXiv:1802.09127* (2018).

[32] Pan Xu et al. 'Neural contextual bandits with deep representation and shallow exploration'. In: *arXiv preprint arXiv:2012.01780* (2020).

[33] Yuta Saito et al. 'Open bandit dataset and pipeline: Towards realistic and reproducible off-policy evaluation'. In: *arXiv preprint arXiv:2008.07146* (2020).

[34] Chongming Gao et al. *KuaiRec: A Fully-observed Dataset for Recommender Systems (Density: Almost 100%)*. Image adapted from KuaiRec site (cropped to remove whitespace). CC BY-SA 4.0. 2026. URL: https://kuairec.com/ (visited on 01/02/2026).

[35] Jiaqi Ma et al. 'Off-policy learning in two-stage recommender systems'. In: *Proceedings of The Web Conference 2020*. 2020, pp. 463–473.

[36] Marius Kaminskas and Derek Bridge. 'Diversity, serendipity, novelty, and coverage: a survey and empirical analysis of beyond-accuracy objectives in recommender systems'. In: *ACM Transactions on Interactive Intelligent Systems (TiiS)* 7.1 (2016), pp. 1–42.

[37] Tomislav Duricic et al. 'Beyond-accuracy: a review on diversity, serendipity, and fairness in recommender systems based on graph neural networks'. In: *Frontiers in big data* 6 (2023), p. 1251072.

[38] F Maxwell Harper and Joseph A Konstan. 'The movielens datasets: History and context'. In: *Acm transactions on interactive intelligent systems (tiis)* 5.4 (2015), pp. 1–19.

[39] Eugene Ie et al. 'Recsim: A configurable simulation platform for recommender systems'. In: *arXiv preprint arXiv:1909.04847* (2019).

[40] Kesen Zhao et al. 'KuaiSim: A comprehensive simulator for recommender systems'. In: *Advances in Neural Information Processing Systems* 36 (2023), pp. 44880–44897.

[41] Ji Yang et al. 'Mixed negative sampling for learning two-tower neural networks in recommendations'. In: *Companion proceedings of the web conference 2020*. 2020, pp. 441–447.

[42] Pallets. *Welcome to Click — Click Documentation (8.3.x)*. 2026. URL: https://click.palletsprojects.com/en/stable/ (visited on 19/03/2026).

[43] Jack Sherman and Winifred J Morrison. 'Adjustment of an inverse matrix corresponding to a change in one element of a given matrix'. In: *The Annals of Mathematical Statistics* 21.1 (1950), pp. 124–127.

[44] Nicholas J Higham. 'Analysis of the Cholesky decomposition of a semi-definite matrix'. In: *Reliable Numerical Computation* (1990).

[45] Paul E Johnson. 'Where do Multivariate Normal Samples Come from?' In: *Center for Research Methods and Data Analysis* (2017).

[46] Minmin Chen et al. 'Values of user exploration in recommender systems'. In: *Proceedings of the 15th acm Conference on recommender systems*. 2021, pp. 85–95.

[47] Pedro R Pires et al. 'Exploitation Over Exploration: Unmasking the Bias in Linear Bandit Recommender Offline Evaluation'. In: *Proceedings of the Nineteenth ACM Conference on Recommender Systems*. 2025, pp. 736–745.

[48] Shangsong Liang et al. 'A survey on variational autoencoders in recommender systems'. In: *ACM Computing Surveys* 56.10 (2024), pp. 1–40.

Appendix A

Two-tower flags and validation plots

A.1 Model flags

```
{
    "save_name": "default",
    "watch_threshold": 2.0,
    "metric_k": [10, 50, 100, 200],
    "best_k": 50,
    "side_features": true,
    "skip_towers": false,
    "hidden_size": [512, 256],
    "output_size": 32,
    "relu": true,
    "dropout": 0.2,
    "l2_norm": true,
    "logit_temp": 0.07,
    "batch_size": 1024,
    "epochs": 100,
    "num_negatives": 1024,
    "negative_sampling": "uniform",
    "weighted_loss": false,
    "popsample_coeff": 1.0,
    "wr_band_ratio": [0.15, 0.25, 0.25, 0.2, 0.15],
    "score_sharpness": 1.0,
    "lr": 0.001,
    "weight_decay": 0.01,
    "optimiser": "adamw",
    "data_workers": 4,
    "id_emb_dims": 32,
    "item_cat_emb_dims": 16,
    "user_cat_emb_root": "2",
    "user_cat_emb_cap": 16,
    "seed": 117
}
```

Figure A.1: `train-tt` flags for model `default`, as JSON

Model	**CLI flag overrides vs.** `default`
`default`	—
`narrower`	`--hidden-size 128 --hidden-size 64`
`wider`	`--hidden-size 1024 --hidden-size 512`
`deeper`	`--hidden-size 512 --hidden-size 256 --hidden-size 128`
`shallower`	`--hidden-size 512`
`in-batch-neg`	`--negative-sampling in-batch`
`user-uniform-neg`	`--negative-sampling score-weighted --score-sharpness 0`
`score-weight-neg`	`--negative-sampling score-weighted`
`wr-banded-neg`	`--negative-sampling watch-ratio`
`pop-weight-neg`	`--negative-sampling popularity`
`full-softmax-neg`	`--negative-sampling full-softmax`
`no-sidefeats`	`--no-side-features`
`no-relu`	`--no-relu`
`id-only-mf`	`--skip-towers --no-side-features`
`no-l2`	`--no-l2-norm`
`weighted-loss`	`--weighted-loss`
`dim-16`	`--output-size 16`
`dim-64`	`--output-size 64`

Table A.1: `train-tt` flag default overrides for all models

A.2 Validation plots

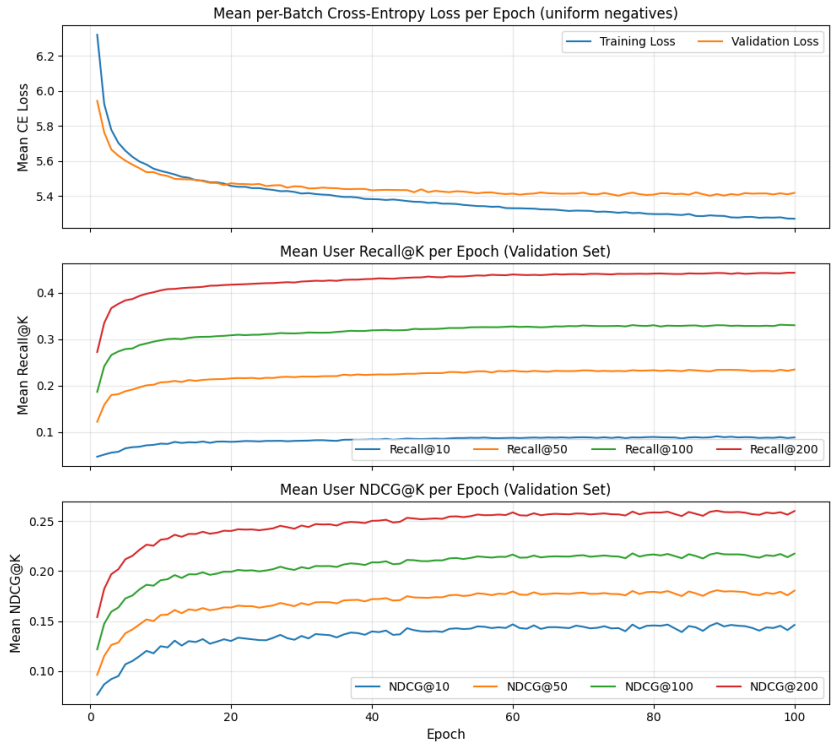

Figure A.2: Train/val metrics for model `default`

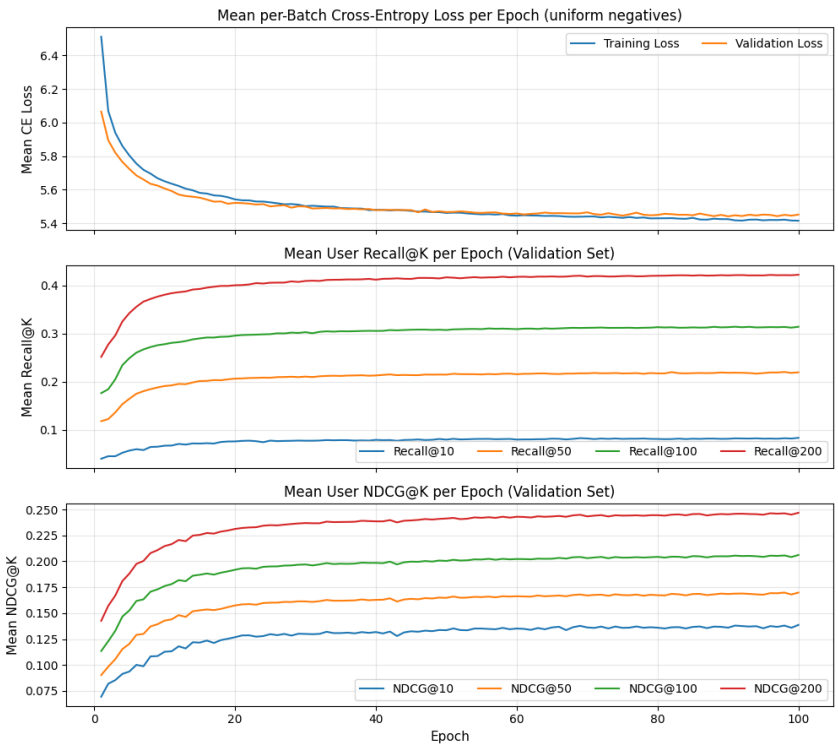

Figure A.3: Train/val metrics for model `narrower`

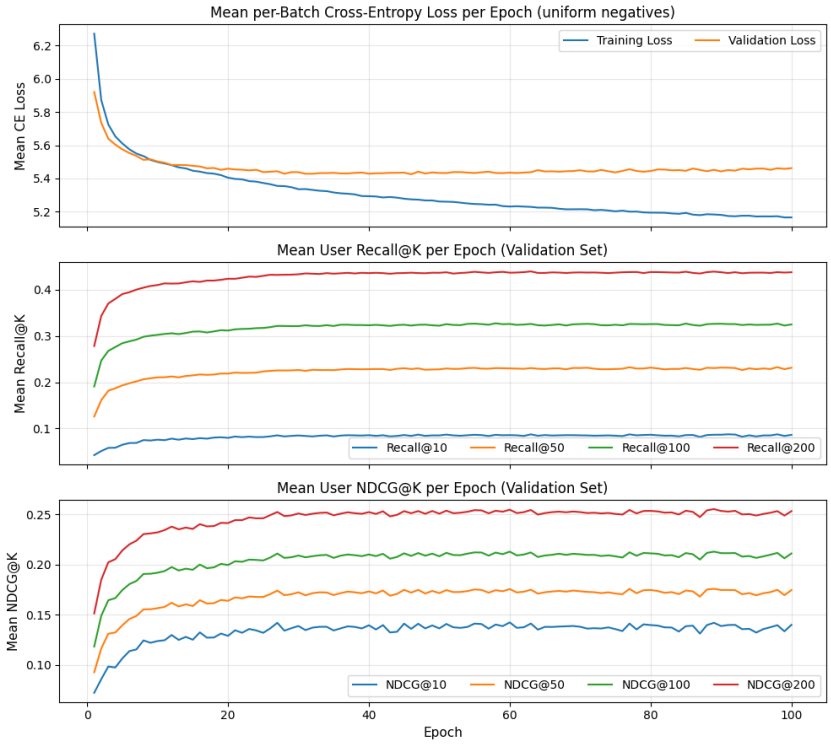

Figure A.4: Train/val metrics for model `wider`

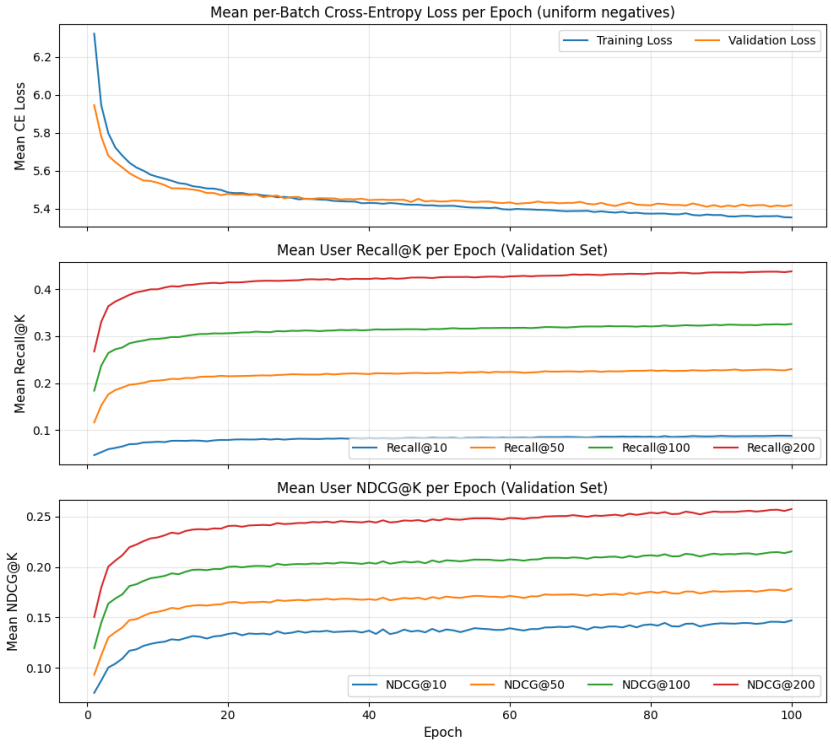

Figure A.5: Train/val metrics for model `deeper`

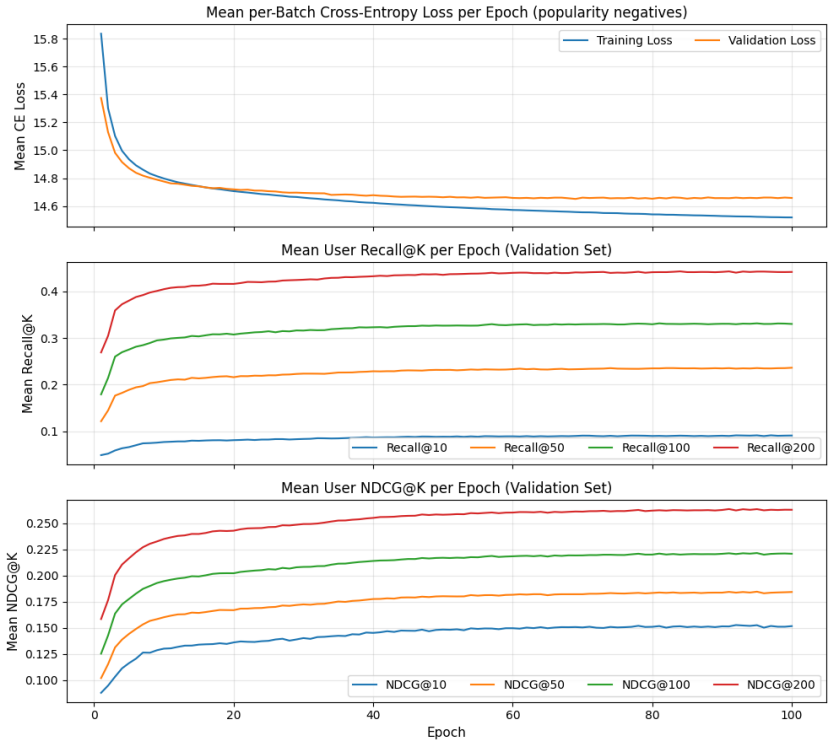

Figure A.6: Train/val metrics for model `shallower`

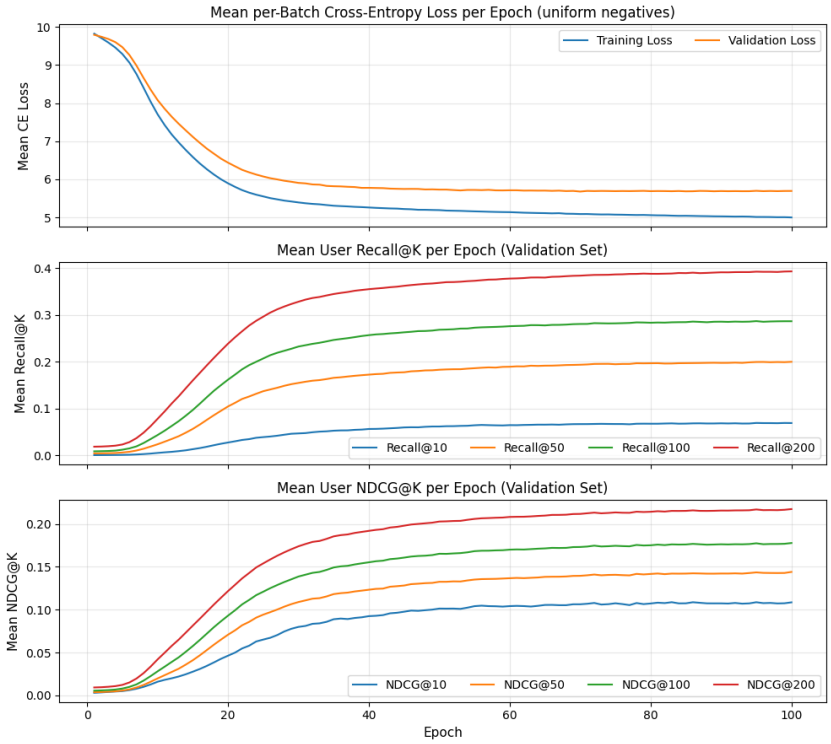

Figure A.7: Train/val metrics for model `in-batch-neg`

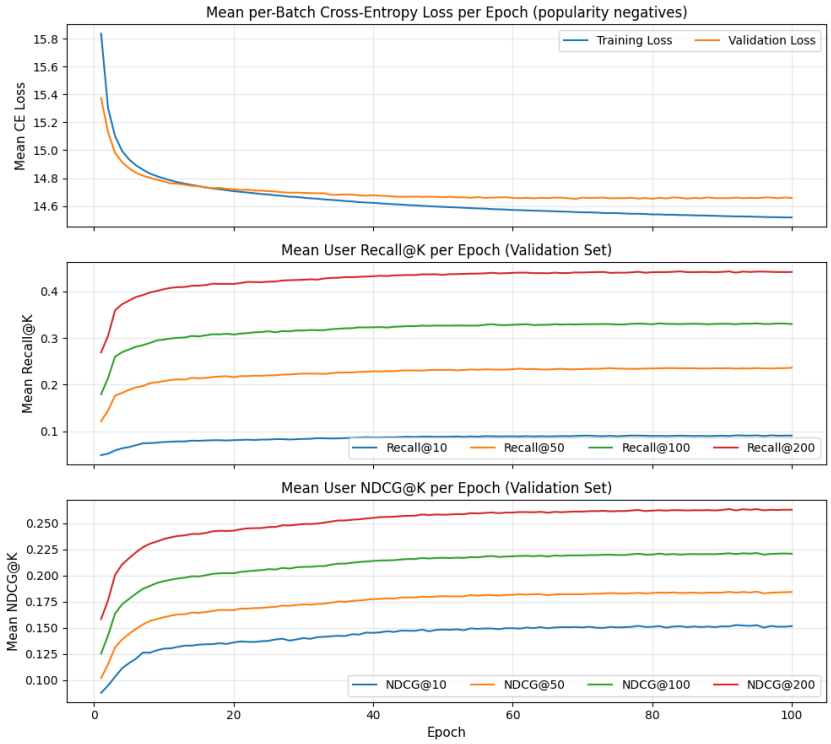

Figure A.8: Train/val metrics for model `user-uniform-neg`

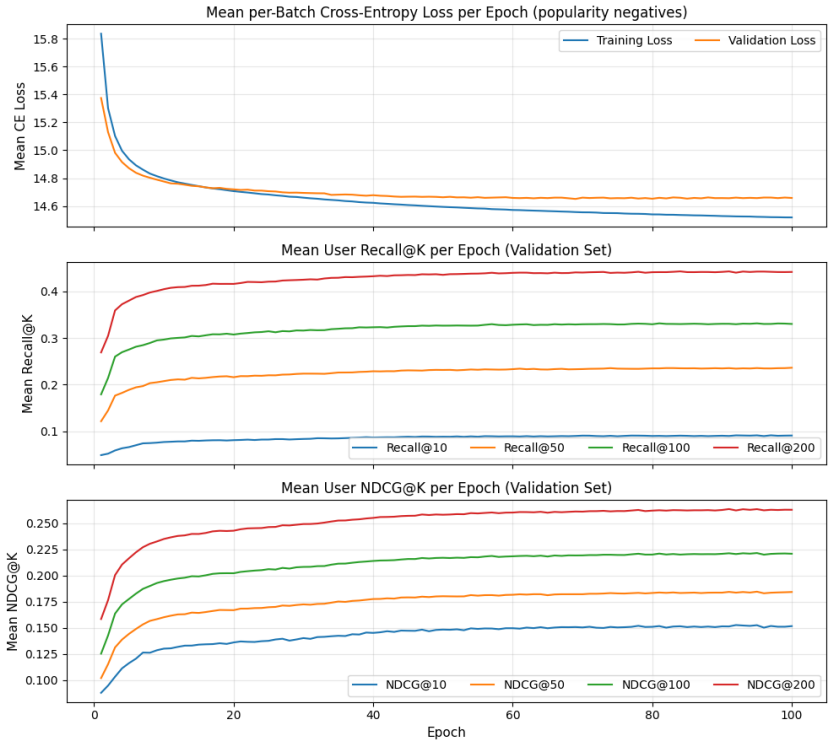

Figure A.9: Train/val metrics for model `score-weight-neg`

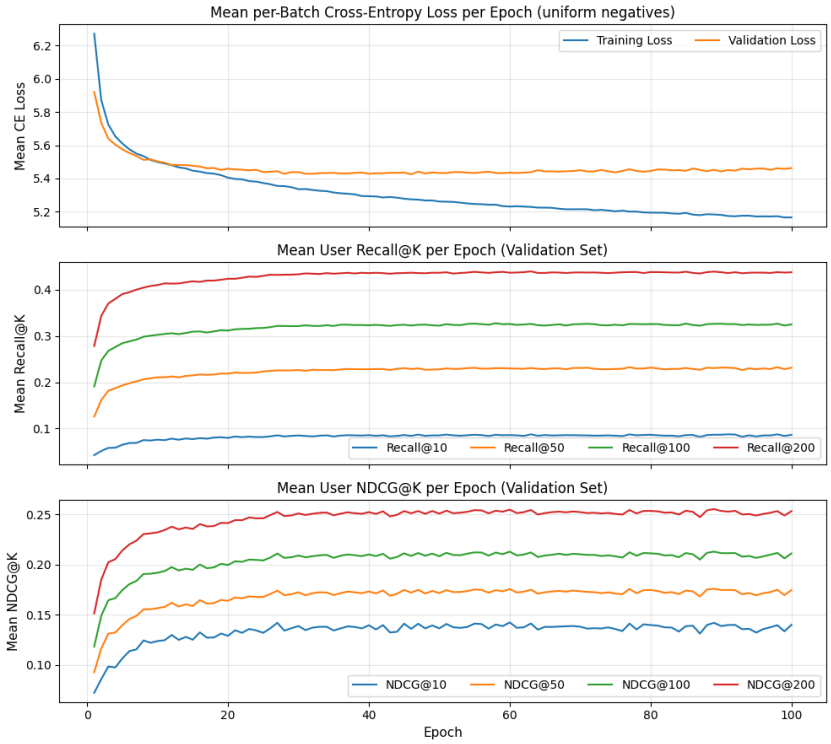

Figure A.10: Train/val metrics for model `wr-banded-neg`

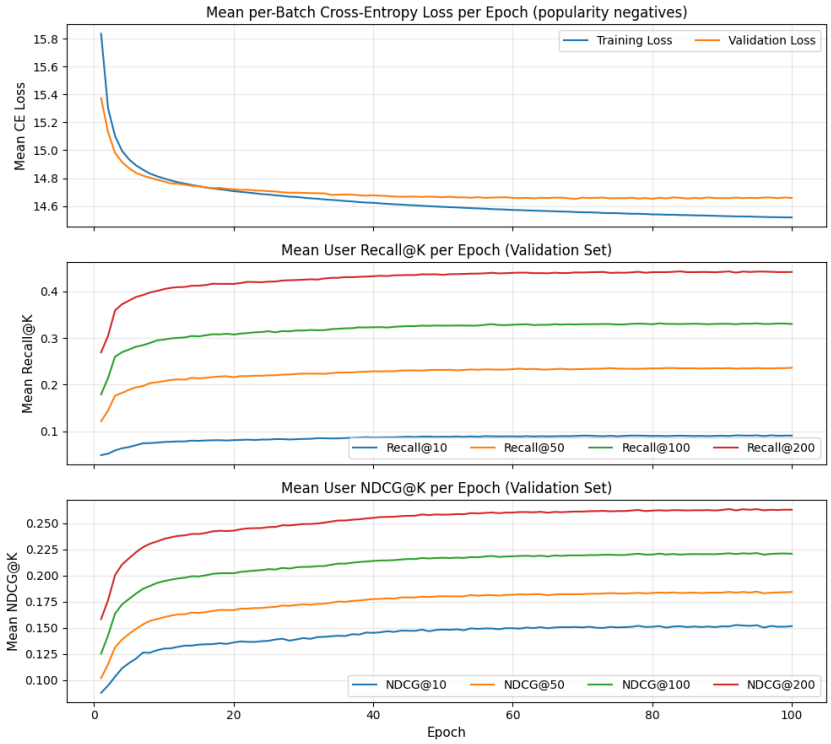

Figure A.11: Train/val metrics for model `pop-weight-neg`

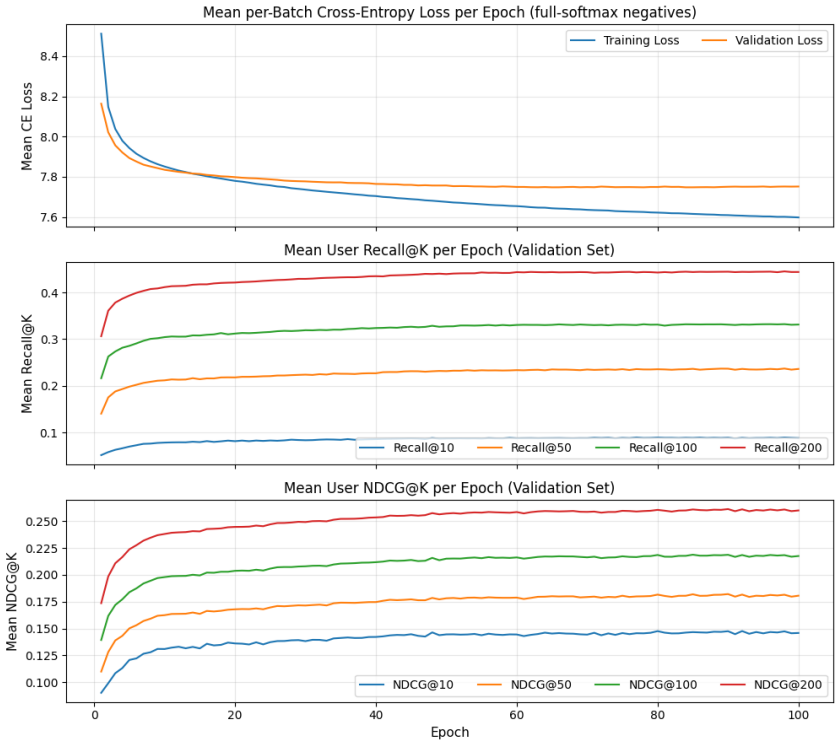

Figure A.12: Train/val metrics for model `full-softmax-neg`

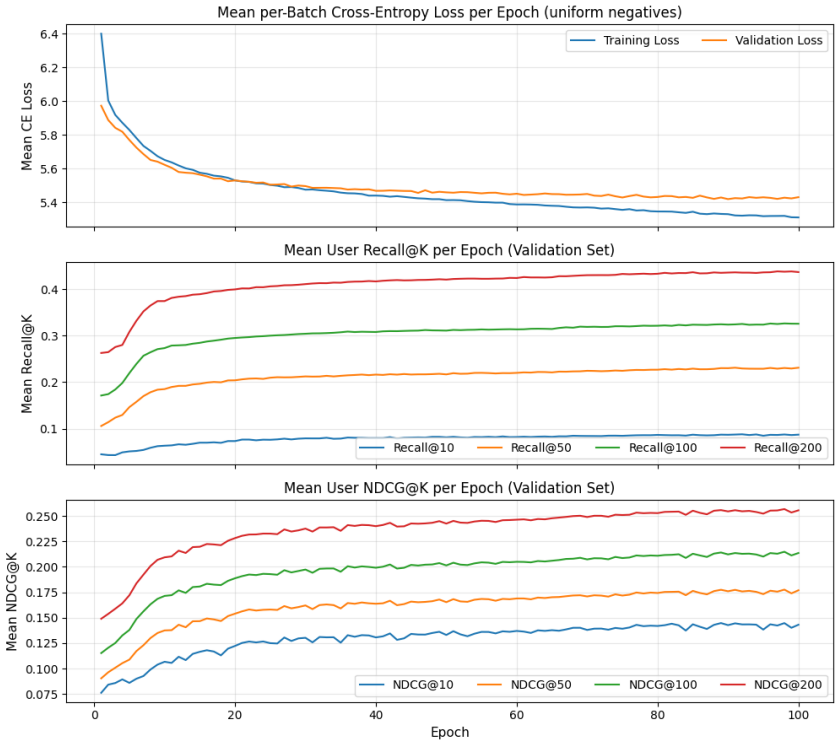

Figure A.13: Train/val metrics for model `no-sidefeats`

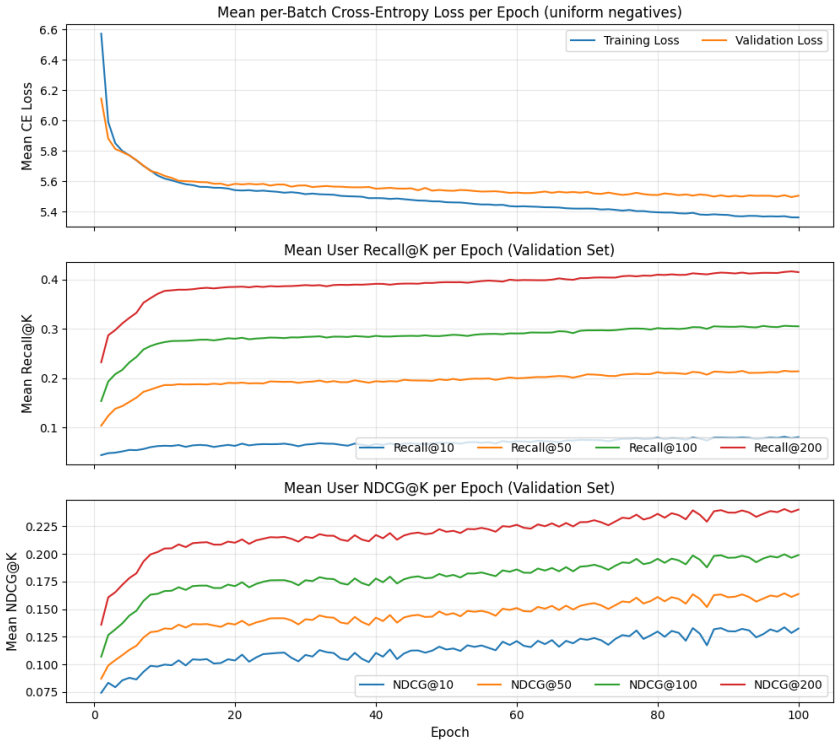

Figure A.14: Train/val metrics for model `no-relu`

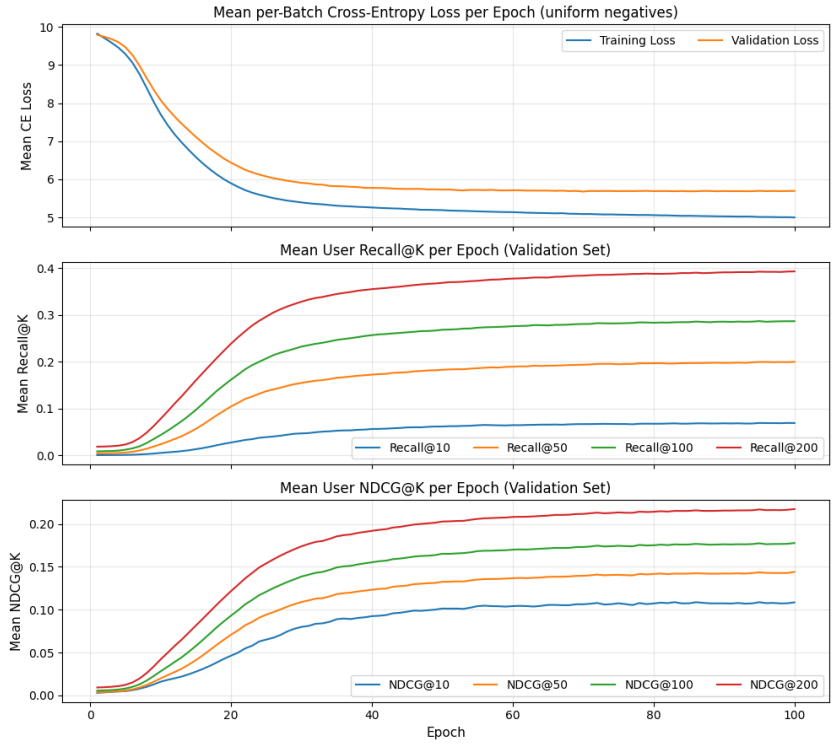

Figure A.15: Train/val metrics for model `id-only-mf`

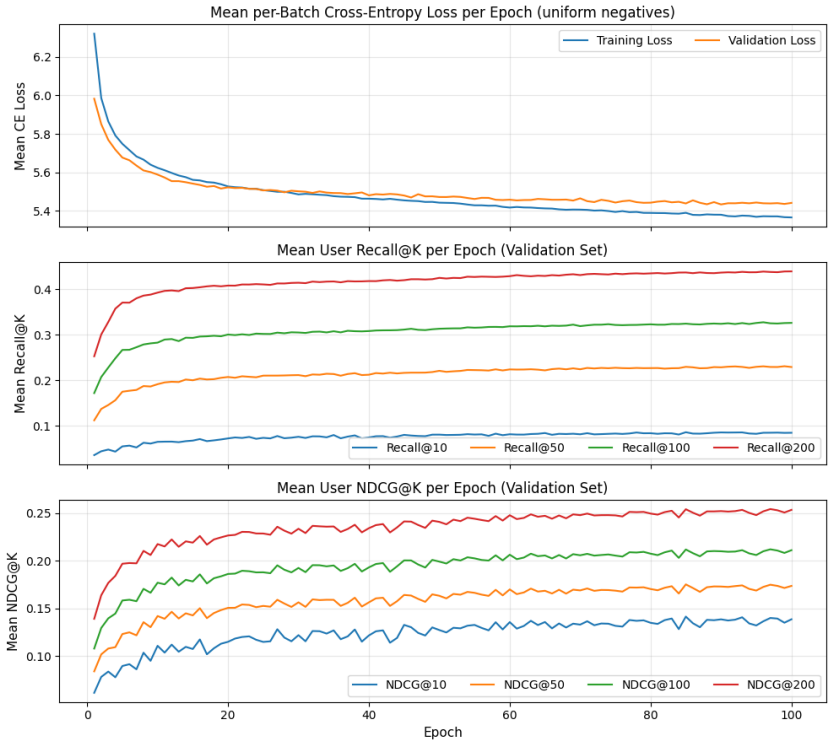

Figure A.16: Train/val metrics for model `no-12`

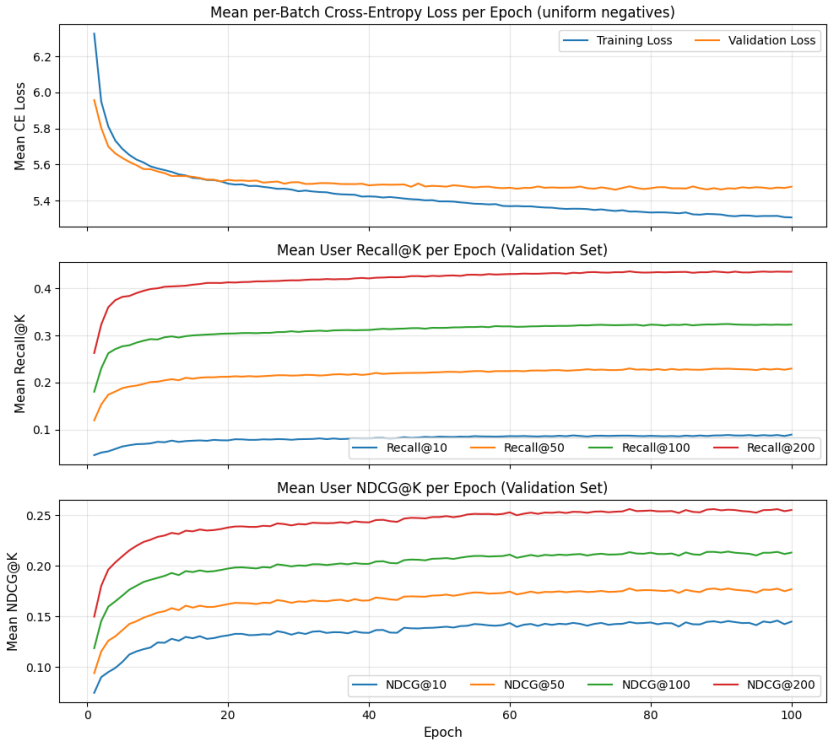

Figure A.17: Train/val metrics for model `weighted-loss`

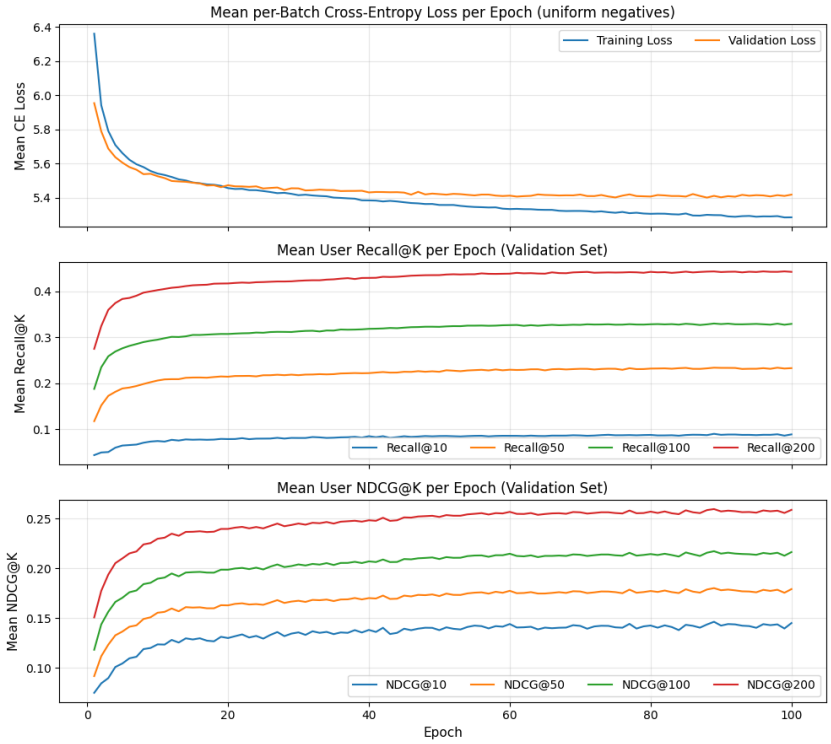

Figure A.18: Train/val metrics for model `dim-16`

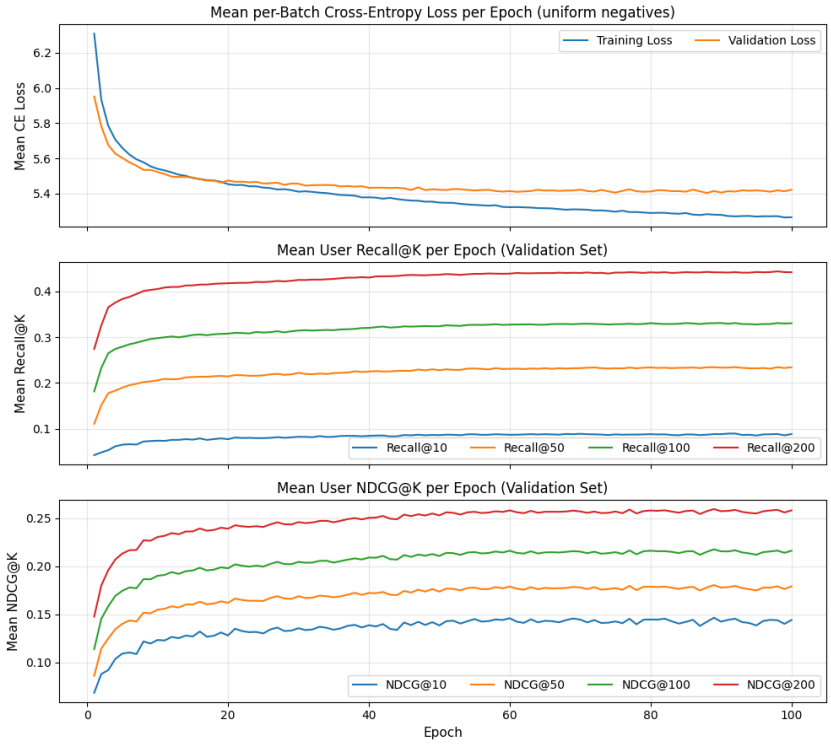

Figure A.19: Train/val metrics for model `dim-64`